

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ФАКУЛЬТЕТ ПРИКЛАДНОЙ МАТЕМАТИКИ И ИНФОРМАТИКИ
Кафедра многопроцессорных систем и сетей

КИРИЕВИЧ
Антон Иванович

РАЗРАБОТКА СИСТЕМЫ ПОДБОРА ОПТИМАЛЬНОГО
МЕСТА РАБОТЫ

Дипломная работа

Научный руководитель:
старший преподаватель
О. М. Кондратьева

Допущен к защите

«___» _____ 2024 г.

Зав. кафедрой многопроцессорных систем и сетей,
кандидат физико-математических наук,
доцент И. Е. Андрушкевич

Минск, 2024

РЕФЕРАТ

Дипломная работа, 48 стр., 39 иллюстр., 2 табл., 12 источн.

NLP, CNN, RNN, LSTM, BERT, GLOVE, BPE, WORDPIECE, HEADHUNTER API, ТЕЛЕГРАМ-БОТ.

Объекты исследования – нейронные сети для работы с текстовыми данными.

Цель исследования – разработка модели предсказания зарплаты вакансии на основе ее текстовых и категориальных признаков.

Методы исследования – системный подход, изучение соответствующей литературы и электронных источников, постановка задачи и её решение.

В результате исследования разработан телеграм-бот, позволяющий предсказать зарплату и найти оптимальное место работы.

Области применения: разработанные методы предсказания зарплаты можно использовать в любых регрессионных задачах, которые используют текстовые и категориальные признаки, разработанный телеграм-бот можно использовать для определения оптимального места работы.

РЭФЕРАТ

Дыпломная работа, 48 старонак, 39 малюнкаў, 2 табл., 12 крыніц.

NLP, CNN, RNN, LSTM, BERT, GLOVE, BPE, WORDPIECE, HEADHUNTER API, ТЭЛЕГРАМ-БОТ.

Аб'екты даследавання – нейронавыя сеткі для працы з тэкставымі дадзенымі.

Мэта даследавання – распрацоўка мадэлі прадказання зарплаты вакансіі на аснове яе тэкставых і катэгарыяльных прыкмет.

Метады даследавання – сістэмны падыход, вывучэнне адпаведнай літаратуры і электронных крыніц, фармуляванне задачы і яе рашэнне.

У выніку даследавання распрацаваны тэлеграм-бот, які дазваляе прадказаць зарплату і знайсці аптымальнае месца працы.

Вобласці прымянення: распрацаваныя метады прадказання зарплаты можна выкарыстоўваць у любых рэгрэсійных задачах, якія выкарыстоўваюць тэкставыя і катэгарыяльныя прыкметы, распрацаваны тэлеграм-бот можна выкарыстоўваць для вызначэння аптымальнага месца працы.

ABSTRACT

Diploma, 48 pages, 39 figures, 2 tables, 12 source.

NLP, CNN, RNN, LSTM, BERT, GLOVE, BPE, WORDPIECE, HEADHUNTER API, TELEGRAM-BOT.

Research Objects – neural networks for working with text data.

Research Objective – development of a model for predicting the salary of a vacancy based on its text and categorical features.

Research Methods – systematic approach, study of relevant literature and electronic sources, problem formulation and solution.

As a result of the research, a telegram bot has been developed that allows you to predict your salary and find the optimal place to work.

Areas of Application: the developed methods for predicting salaries can be used in any regression tasks that use text and categorical features; the developed telegram bot can be used to determine the optimal place of work.

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	6
ГЛАВА 1 ВВЕДЕНИЕ В НЕЙРОННЫЕ СЕТИ	8
1.1 Полносвязные нейронные сети.	8
1.2 Сверточные нейронные сети	11
1.3 Нейронные сети для работы с последовательностями	14
1.3.1 Эмбединги слов	14
1.3.2 Рекуррентные нейронные сети	17
1.4 Трансформеры.	19
1.4.1 Механизм внутреннего внимания	19
1.4.2 Архитектура трансформера	21
ГЛАВА 2 РЕШЕНИЕ ИСХОДНОЙ ЗАДАЧИ	26
2.1 Датасет Adzuna	26
2.1.1 Предобработка данных	26
2.1.2 Архитектуры моделей	28
2.1.3 Обучение моделей	32
2.1.4 Результаты	33
2.2 Датасет rabota.by	35
2.2.1 Сбор данных	35
2.2.2 Предобработка данных	36
2.2.3 Архитектуры моделей	39
2.2.4 Обучение моделей	39
2.2.5 Результаты	40
2.2.6 Разработка телеграм-бота	42
ЗАКЛЮЧЕНИЕ	46
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	48

ВВЕДЕНИЕ

В современном мире, где информация играет ключевую роль, поиск оптимального места работы является важной задачей для многих людей. Однако, среди множества доступных вакансий, выбор той, которая соответствует нашим ожиданиям и предоставляет достойную зарплату, может быть сложной задачей. В таком контексте, разработка системы, способной предсказывать зарплату по описанию вакансии, имеет огромный потенциал для помощи соискателям в поиске и выборе работы, наиболее соответствующей их требованиям и ожиданиям.

Обработка текстов на естественном языке (Natural Language Processing, NLP) — общее направление искусственного интеллекта и математической лингвистики. Оно изучает проблемы компьютерного анализа и синтеза текстов на естественных языках.

Сегодня NLP становится все более важным направлением в области искусственного интеллекта и машинного обучения. Благодаря развитию глубокого обучения и нейронных сетей, стали доступны новые методы анализа и обработки текстов, которые позволяют получать более точные результаты в различных задачах. Применение нейронных сетей для анализа текстовых признаков уже показало впечатляющие результаты во многих задачах NLP, включая классификацию текстов, машинный перевод, распознавание речи и многое другое.

Целью данной работы является разработка системы подбора оптимального места работы, которая на основе анализа текстовых и категориальных признаков сможет предсказывать зарплату по описанию вакансии. Для решения данной задачи мы будем исследовать и применять различные методы NLP, включая нейронные сети.

В первой главе мы рассмотрим основы нейронных сетей, которые являются основным инструментом в нашей работе. Мы ознакомимся с полносвязными нейронными сетями, которые применяются для обработки категориальных признаков, и исследуем сверточные нейронные сети, которые, несмотря на то, что создавались для решения задач, связанных с изображениями, также позволяют анализировать текстовые признаки. Также будут рассмотрены нейросети для работы с последовательностями, такие как рекуррентные нейронные сети (RNN) и трансформеры, которые специально разработаны для обработки текстовых данных.

Вторая глава будет посвящена разработке решения для предсказания

зарплаты на основе описания вакансии. Мы начнем с предобработки данных, включающей очистку и токенизацию текстовых признаков, а также кодирование категориальных признаков. Затем мы опишем архитектуры используемых моделей. Обучение моделей будет осуществляться на размеченных данных, используя методы глубокого обучения. В разделе результатов мы проанализируем точность предсказаний наших моделей.

Для применения полученных моделей на практике разработаем телеграм-бот, который позволит предсказать заработную плату указанной вакансии и, соответственно, найти оптимальное место работы.

ГЛАВА 1

ВВЕДЕНИЕ В НЕЙРОННЫЕ СЕТИ

Для предсказания заработной платы мы будем использовать различные нейросетевые архитектуры. Однако прежде чем вдаваться в детали архитектуры модели, нужно понять, что такое нейросеть, какие основные архитектуры существуют, и как ее обучить конкретно для нашей задачи. Для начала дадим определение нейронной сети:

Нейронная сеть — это сложная дифференцируемая функция, задающая отображение из исходного признакового пространства в пространство ответов, все параметры которой могут настраиваться одновременно и взаимосвязанно. В частном (и наиболее частом) случае представляет собой последовательность дифференцируемых параметрических преобразований [1].

1.1 Полносвязные нейронные сети

Полносвязные нейронные сети (Fully Connected Neural Networks) являются одной из наиболее распространенных и базовых архитектур нейронных сетей. Нейронные сети состоят из нейронов, объединенных в слои. Конкретно в полносвязных нейронных сетях каждый нейрон в предыдущем слое связан с каждым нейроном в следующем слое, что обеспечивает полную связность между слоями. Эти связи имеют соответствующие веса, которые подлежат обучению. Пример того, как может выглядеть полносвязная нейронная сеть, изображен на рисунке 1.1.

Основная идея работы нейронных сетей заключается в передаче информации от входных данных через промежуточные слои до выходного слоя. Каждый нейрон в слое получает входные данные, умножает их на соответствующие веса и применяет активационную функцию к полученному взвешенному суммарному значению. Активационная функция обычно введена для внесения нелинейности в модель и позволяет моделировать более сложные зависимости в данных. Таким образом, нейронная сеть — это функция, состоящая простых блоков:

Схема работы нейросети

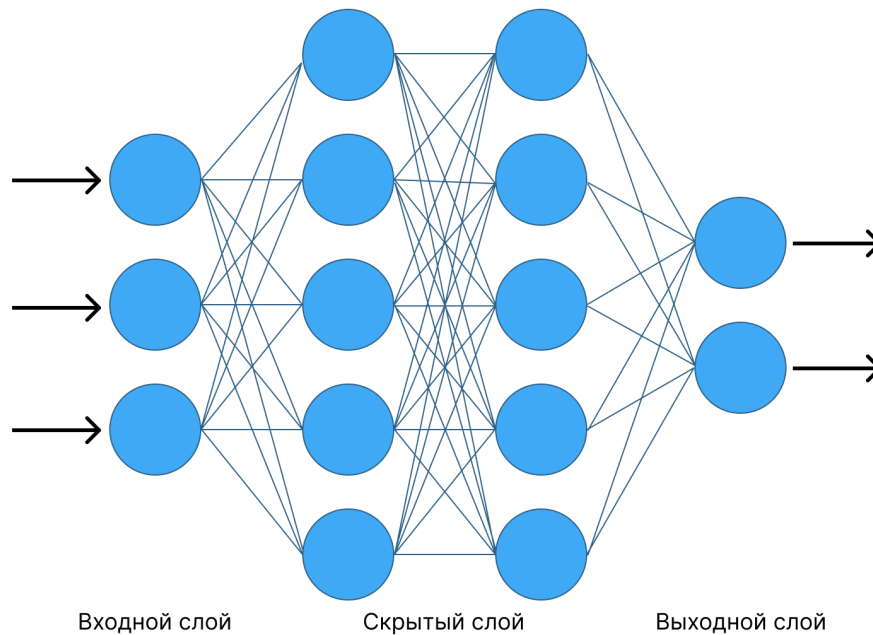


Рисунок 1.1 – Полносвязная нейронная сеть

1. Линейный слой (linear layer, dense layer) — линейное преобразование над входящими данными (его обучаемые параметры — это матрица W и вектор b): $x \rightarrow xW + b$ ($W \in \mathbb{R}^{d \times k}, x \in \mathbb{R}^d, b \in \mathbb{R}^k$). Такой слой преобразует d -мерные векторы в k -мерные.
2. Функция активации (activation function) — нелинейное преобразование, поэлементно применяющееся к пришедшим на вход данным. Благодаря функциям активации нейронные сети способны порождать более информативные признаковые описания, преобразуя данные нелинейным образом. Может использоваться, например, ReLU (rectified linear unit, $ReLU(x) = \max(0, x)$) или, например, сигмоида $\sigma(x) = \frac{1}{1+e^{-x}}$.

Следует также упомянуть, что в литературе нет единства терминологии для определения слоя нейронной сети. Так, например, слоем можно назвать композицию линейного слоя и функции активации, поскольку чаще всего линейные слои не используются без нелинейности. В ряде источников слоями могут также называть все промежуточные представления.

Применение нейронной сети к данным называют прямым проходом (forward pass). Существует также обратный проход (back propagation, backward pass), на котором основан процесс обучения нейронных сетей. Backward pass позволяет настраивать веса в нейронной сети, минимизируя ошибку между предсказанными значениями и ожидаемыми значениями. Во

время обучения, данные подаются на вход сети, и для каждого примера вычисляются предсказания. Затем сравниваются предсказания с правильными ответами, и с помощью backward pass вычисляются градиенты весов. С помощью градиентов обновляются веса нейронов, чтобы уменьшить ошибку предсказания. Обновление весов может происходить, например, с помощью стохастического градиентного спуска (stochastic gradient descent, SGD). Backpropagation считает градиенты оптимальным образом с помощью так называемого chain rule (переиспользуя уже посчитанные градиенты и значения, посчитанные на forward pass):

$$f(w_0) = g_m(g_{m-1}(\dots g_1(w_0) \dots))$$

$$f'(w_0) = g'_m(g_{m-1}(\dots g_1(w_0) \dots)) \cdot g'_{m-1}(g_{m-2}(\dots g_1(w_0) \dots)) \cdot \dots \cdot g'_1(w_0)$$

Обновление весов с помощью типичного алгоритма SGD может выглядеть следующим образом:

$$w_{k+1} = w_k - \alpha \nabla f(x_k)$$

Обновление происходит до сходимости функции потерь к локальному минимуму.

Полносвязные нейронные сети обладают способностью моделировать сложные функции, что доказывает следующая теорема:

Теорема Цыбенко (1989). Для любой непрерывной функции $f(x) : \mathbb{R}^m \rightarrow \mathbb{R}$ и для любого $\varepsilon > 0$ найдется число N , а также числа $w_1, \dots, w_N, b_1, \dots, b_N, \alpha_1, \dots, \alpha_n$, для которых выполняется:

$$\left| f(x) - \sum_{i=1}^N \alpha_i \sigma(\langle x, w_i \rangle + b_i) \right| < \varepsilon, \forall x \in [0, 1]^m$$

Однако, при работе с текстовыми данными или последовательностями, полносвязные нейронные сети могут иметь ограниченные возможности в выявлении скрытых зависимостей в тексте.

В последние годы появились более сложные архитектуры нейронных сетей, такие как сверточные нейронные сети, рекуррентные нейронные сети и трансформеры, которые лучше подходят для работы с текстовыми данными. Однако, полносвязные нейронные сети остаются важным инструментом и могут использоваться в сочетании с другими архитектурами для решения различных задач.

1.2 Сверточные нейронные сети

Сверточные нейронные сети (Convolutional Neural Networks, CNN) являются особой архитектурой нейронных сетей, которая показывает неплохие результаты в обработке и анализе данных с пространственной или сетевой структурой, таких как изображения, звуковые волны или тексты. Наибольшую популярность сверточные нейронные сети приобрели в задачах, связанных с обработкой изображений, таких как, например, распознавание образов на изображениях, классификация изображений и т.д.

Основная идея сверточных нейронных сетей заключается в использовании операции свертки для обнаружения локальных шаблонов или признаков во входных данных. Каждый слой сверточной нейронной сети состоит из нескольких фильтров (ядер), которые скользят по входным данным и вычисляют свертку между фильтром и соответствующей областью входных данных. Затем результаты свертки суммируются для создания нового представления данных. Схема работы операции свертки представлена на рисунке 1.2.

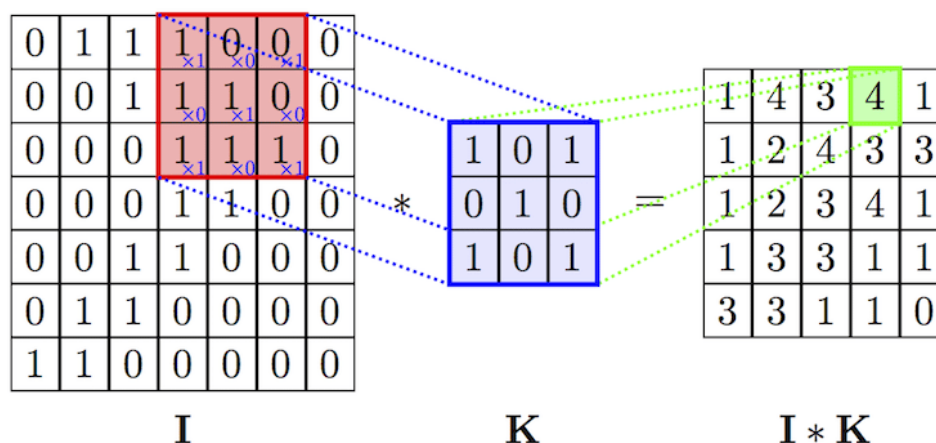
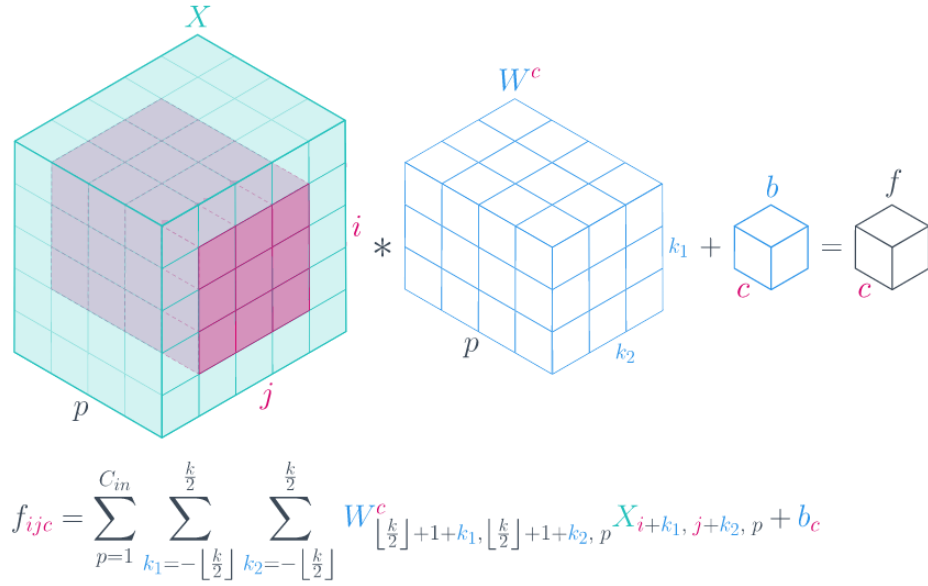


Рисунок 1.2 – Схема работы операции свертки

На рисунке 1.3 можно увидеть, как более формально для многоканального входа выглядит операция свертки.

k — размер ядра; X — карта признаков $H \times W \times C_{in}$; f — карта признаков $H \times W \times C_{out}$



$$f_{ijc} = \sum_{p=1}^{C_{in}} \sum_{k_1=-\lfloor \frac{k}{2} \rfloor}^{\lfloor \frac{k}{2} \rfloor} \sum_{k_2=-\lfloor \frac{k}{2} \rfloor}^{\lfloor \frac{k}{2} \rfloor} W^c_{\lfloor \frac{k}{2} \rfloor + 1 + k_1, \lfloor \frac{k}{2} \rfloor + 1 + k_2, p} X_{i+k_1, j+k_2, p} + b_c$$

Рисунок 1.3 — Формальное определение операции свертки для многоканального входа

Ключевое преимущество сверточных нейронных сетей заключается в их способности распознавать локальные шаблоны независимо от их положения во входных данных. Это достигается путем использования общих весов для всех областей входных данных, с которыми происходит свертка. Таким образом, нейронная сеть обучается распознавать одну и ту же фичу в разных частях входных данных. Если сравнивать с полносвязными нейронными сетями, то для них нам пришлось бы многоканальный вход представить в виде вектора, т.е. если у нас сеть будет в таком случае глубокой, то у нас будет слишком много параметров (которые еще должны обучиться, чтобы уловить пространственную инвариантность), т.е. так полносвязная нейронная сеть рискует переобучиться, не выучив необходимые паттерны, в случае же малого количества параметров полносвязная нейронная сеть ничего особенного не выучит. Таким образом, сверточные нейронные сети имеют явное преимущество по сравнению с полносвязными нейронными сетями.

Следует также отметить, что back propagation для сверточных слоев также можно описать с помощью операции свертки [2], как на рисунке 1.4.

$$\frac{\partial L}{\partial F} = \text{Convolution} \left(\text{Input } X, \text{ Loss gradient } \frac{\partial L}{\partial O} \right)$$

$$\frac{\partial L}{\partial X} = \text{Full Convolution} \left(\begin{matrix} 180^\circ \text{rotated} \\ \text{Filter } F \end{matrix}, \text{ Loss Gradient } \frac{\partial L}{\partial O} \right)$$

Рисунок 1.4 – Обратное распространение ошибки, записанное с помощью операции свертки

Как уже отмечалось ранее, сверточные нейронные сети полезны и для задач NLP. В NLP часто в качестве представления слова (или токена, токеном может являться часть слова или даже символ) используют соответствующий ему вектор, так называемый эмбединг. Таким образом, текст можно представить как матрицу, состоящую из последовательных эмбедингов токенов этого текста. Т.е. над исходным текстом, как и для обычного изображения, можно выполнить операцию свертки, т.е. построить сверточную нейронную сеть, работающую с текстом.

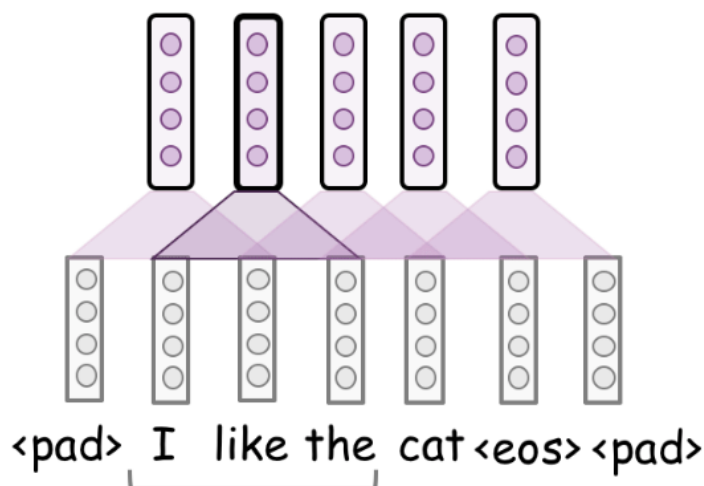


Рисунок 1.5 – Сверточный слой для последовательности эмбедингов текста

Единственное отличие от изображений состоит в том, что здесь будет использоваться одномерная свертка, т.к. в ядро должен попадать весь эмбединг слова. Пример того, как выглядит сверточный слой, работающий с эмбедингами, изображен на рисунке 1.5.

1.3 Нейронные сети для работы с последовательностями

1.3.1 Эмбединги слов

Прежде, чем перейти непосредственно к архитектурам, которые работают с последовательностями (в том числе и с текстами), необходимо понять, как произвольный текст подать на вход нейронной сети. Самые первые подходы для получения представления текста (эмбединга текста) были статистическими, самые известные из которых это "Bag-of-words" (вектор частот встречаемости каждого токена) и TF-IDF [3]. TF-IDF использует не только информацию, содержащуюся в самом тексте, но и пытается соотнести его с остальными текстами из имеющейся коллекции D . Таким образом, представление текста d состоит из произведений $TF(t, d) \cdot IDF(t, D)$ по всем токенам t , где TF и IDF определены по следующим формулам:

$$TF(t, d) = \frac{n_t}{\sum_k n_k}$$

$$IDF(t, D) = \log \frac{|D|}{|d_i \in D | t \in d_i|}$$

В первой формуле n_t — число вхождений токена t в документ, в знаменателе у TF соответственно стоит общее число токенов в данном документе d . Во второй формуле $|d_i \in D | t \in d_i|$ — число документов в текстовой коллекции D , в которых встречается токен t . Таким образом, TF — частота вхождения токена в документ, а IDF — множитель, который штрафует компоненты, отвечающие слишком распространённым токенам, и повышает вес специфических для отдельных текстов (и, вероятно, информативных) слов.

Статистические подходы в некоторых ситуациях могут быть эффективными, однако, поскольку они рассматривают текст как неупорядоченный набор токенов, все же уступают современным нейросетевым подходам. Как уже упоминалось ранее, для представления токенов могут использоваться эмбединги, а для представления всего текста — последовательность этих эмбедингов, соответствующих токенам в тексте. Но отсюда возникает вопрос о вычислении этих эмбедингов. Отсюда же возникает второй вопрос — как заключить в векторе смысл? Нетрудно заметить, что нужно воспользоваться *контекстом*. Одним из реализаций данного подхода является Word2vec, для обучения которого авторы предложили две стратегии: Skip-gram и CBOW (Continuous bag-of-words). В архитектуре CBOW модель учится предска-

вать данное (центральное) слово по контексту (например, по трем словам перед данным и трем словам после него), как на рисунке 1.6.



Рисунок 1.6 – Архитектура CBOW

В архитектуре Skip-gram модель учится по слову предсказывать контекст (например, каждого из трех соседей слева и справа), как на рисунке 1.7.

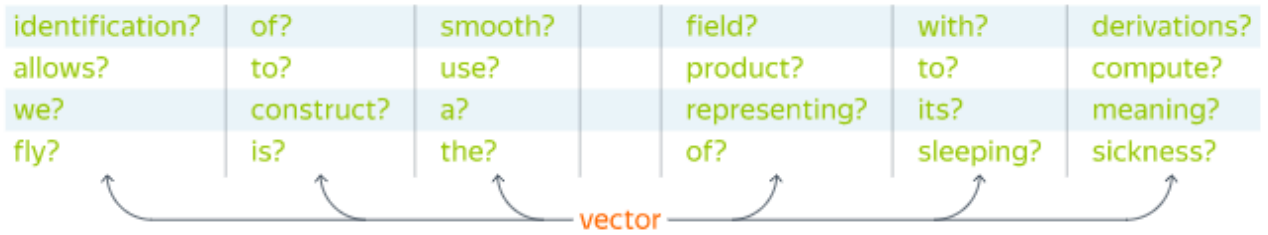


Рисунок 1.7 – Архитектура Skip-gram

Для каждого слова w обучается два отдельных эмбединга: v_u и v_w , где первый эмбединг используется, когда w является центральным, а второй эмбединг используется, когда w рассматривается как часть какого-либо контекста. Конкретно в модели CBOW при фиксированном контексте $context$ вычисляются следующие логиты для всевозможных слов u :

$$logits_u = \langle \sum_{w \in context} v_w, v_u \rangle$$

После этого вычисляются вероятности этих слов u быть центральным в данном контексте $context$ с помощью функции *softmax*:

$$softmax(logits)_u = \frac{e^{logits_u}}{\sum_u e^{logits_u}}$$

Модель учится с помощью SGD на кросс-энтропию полученного распределения с истинным распределением центральных слов. Пример работы модели CBOW можно увидеть на рисунке 1.8.

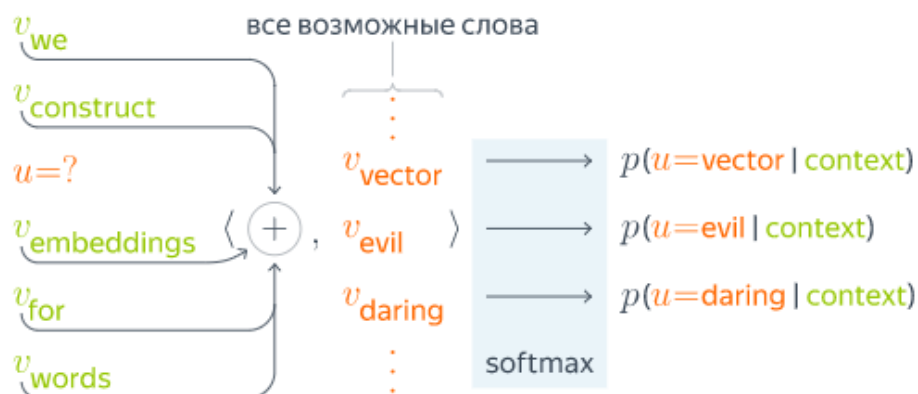


Рисунок 1.8 – Пример работы модели CBOW

В модели Skip-gram по данному центральному слову для каждой позиции контекста *context* независимо предсказывается распределение вероятностей. В качестве функции потерь выступает сумма кросс-энтропий распределений слов контекста с их истинными распределениями. Пример работы модели Skip-gram можно увидеть на рисунке 1.9.

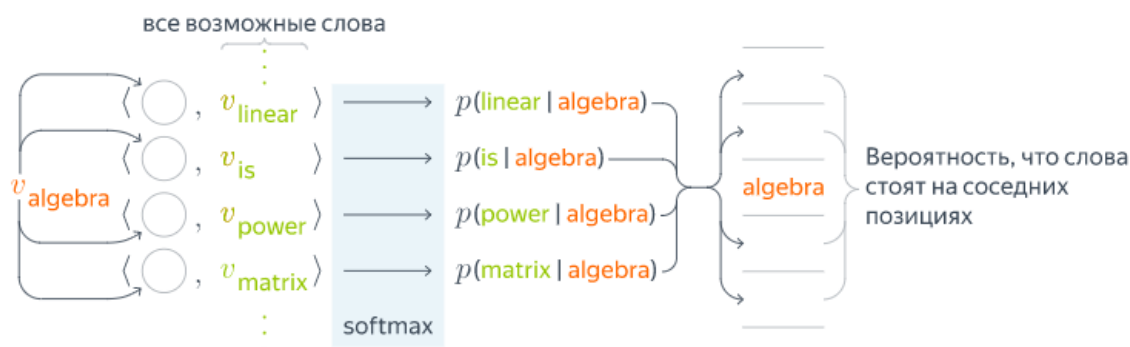


Рисунок 1.9 – Пример работы модели Skip-gram

Существуют и другие реализации данного подхода, такие как GloVe, FastText, ELMo, BERT. Таким образом, наиболее эффективным вариантом для работы с текстом в нашем случае будет взять готовые эмбединги токенов, которые уже обучились на большом количестве текстов и на основе различных контекстов уже содержат в себе какой-то смысл.

1.3.2 Рекуррентные нейронные сети

RNN (Recurrent Neural Network) – это класс нейронных сетей, который предназначен для обработки последовательностей данных, где входные данные представлены в виде последовательности, и каждый элемент последовательности имеет зависимость от предыдущих элементов. RNN обладает своеобразной архитектурой, которая позволяет ей сохранять состояние (память) и использовать его для обработки последующих элементов последовательности. Пример того, как выглядит рекуррентная нейронная сеть, изображен на рисунке 1.10.

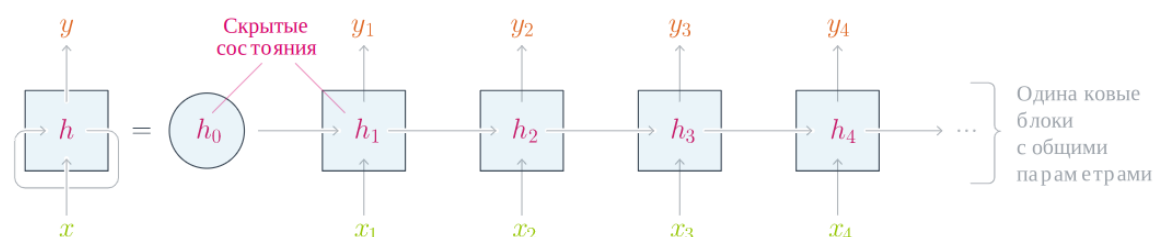


Рисунок 1.10 – Рекуррентная нейронная сеть

Основной компонент RNN – это рекуррентный слой (recurrent layer), состоящий из нейронов, которые имеют обратные связи. Эти обратные связи позволяют передавать информацию от предыдущего элемента последовательности к следующему, образуя своеобразную "внутреннюю память" о предыдущих состояниях. Таким образом, RNN может учитывать контекст и последовательность данных при выполнении задач обработки последовательностей. Более формально, на каждом шаге внутренняя память (скрытое состояние или hidden state, векторы h_n) обновляется, учитывая предыдущее скрытое состояние и приходящий на вход вектор x_n (эмбединг токена), например, следующим образом:

$$h_n = \tanh(h_{n-1}W_1 + x_nW_2)$$

После этого по текущему скрытому состоянию может предсказываться выход следующим образом:

$$y_n = h_nW_3$$

С помощью различных RNN можно получать более глубокие RNN, в которых первый слой RNN будет принимать исходную последовательность эмбедингов, второй слой RNN - выходы первого слоя, третий слой - выходы второго слоя и т.д.. Пример глубокой RNN изображен на рисунке 1.11.

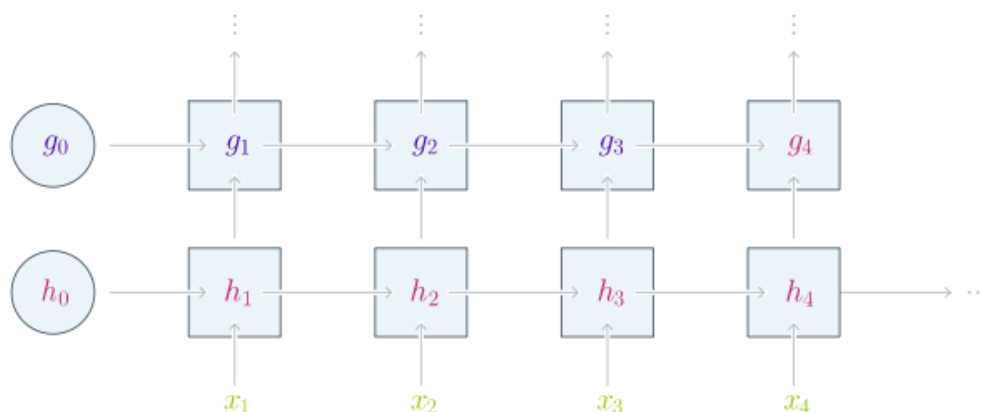


Рисунок 1.11 – Глубокая рекуррентная нейронная сеть

Стандартная RNN учитывает только предыдущий контекст. Однако слово связано не только с предыдущими, но и с последующими словами, поэтому часто также используют двунаправленную рекуррентную сеть (bidirectional RNN, BRNN). В ней есть две рекуррентные подсети, в одной из которых токены идут от первого к последнему, а во второй — от последнего к первому. Пример такой сети изображен на рисунке 1.12.

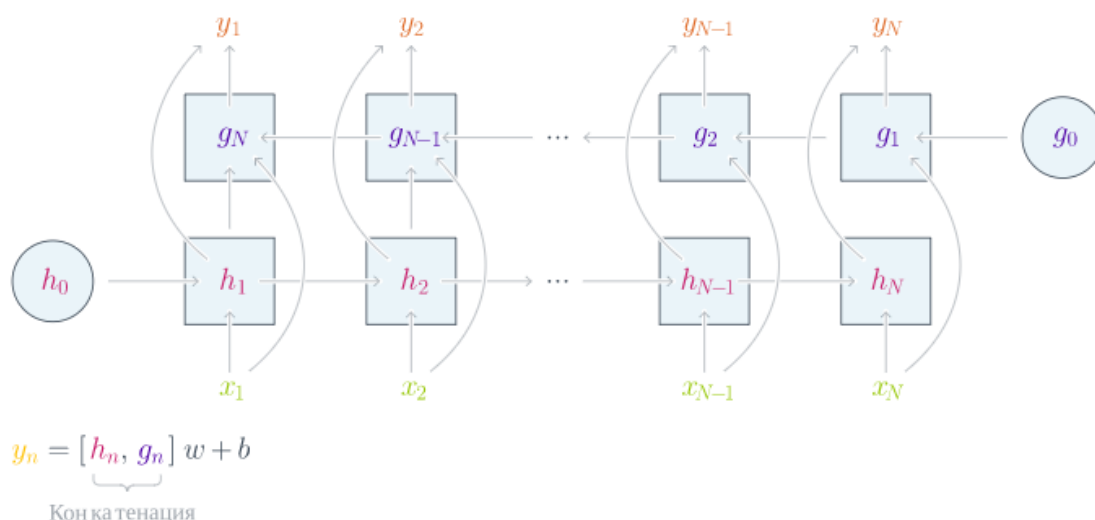


Рисунок 1.12 – Двунаправленная рекуррентная нейронная сеть

Здесь на изображении формула для y_n может быть другой, т.е. по-другому учитывать выходы двух RNN.

Однако, у простейших RNN есть проблема, называемая проблемой затухающего или взрывающегося градиента. При обучении RNN градиенты, передаваемые через обратные связи, могут с течением времени становиться очень маленькими или очень большими, что приводит к проблемам с обуче-

нием и стабильностью модели:

$$\nabla_{h_{i-1}} L = (\nabla_{h_{i-1}} L) W_1^T \odot \tanh'(h_{i-1} W_1 + x_i W_2)$$

Со взрывами градиентов помогает специальный трюк gradient clipping:

$$\|\nabla \mathcal{L}\|_{clip} = \begin{cases} \|\nabla \mathcal{L}\|, & \|\nabla \mathcal{L}\| < \tau \\ \tau, & \|\nabla \mathcal{L}\| \geq \tau \end{cases}$$

Однако проблема с затуханием градиента остается. Для преодоления этой проблемы были разработаны различные модификации RNN, такие как LSTM (Long Short-Term Memory) и GRU (Gated Recurrent Unit). Эти модели имеют более сложную архитектуру, которая позволяет эффективно управлять памятью и обработкой последовательностей, сохраняя более долговременные зависимости в данных.

1.4 Трансформеры

1.4.1 Механизм внутреннего внимания

Механизм внутреннего внимания (self-attention) используется, чтобы посмотреть на другие слова во входной последовательности во время кодирования конкретного слова. В отличие от классических рекуррентных нейронных сетей (RNN), которые последовательно обрабатывают данные, механизм внутреннего внимания позволяет моделировать зависимости между всеми парами элементов в последовательности, независимо от их относительного расположения. Изначально этот механизм был представлен в статье "Attention is all you need" как элемент архитектуры Трансформер (Transformer) [4]. Эффективность Трансформера демонстрировалась на примере задачи машинного перевода. Сейчас Трансформеры и self-attention обрели огромную популярность и используются не только в NLP, но и в других областях.

Опишем, в чем заключается механизм внутреннего внимания. В качестве параметров в данном модуле хранятся три матрицы. Матрица для запросов W_V , матрица для ключей W_K и матрица для значений W_V . С помощью данных матриц мы можем получать соответствующие вектора query, key, value для векторов-эмбеддингов исходного текста. Для каждого исходного эмбеддинга мы находим скалярное произведение его вектора query на вектора key всех остальных эмбеддингов текста, находя таким образом "важность" того или иного токена в тексте в кодировании исходного токена. Данные скаляр-

ные произведения затем можно нормализовать и пропустить через softmax ($softmax(z)_i = e^{z_i} / \sum_{j=1}^n e^{z_j}$), а затем можно посчитать взвешенную сумму value векторов (соответствующих остальным токенам), используя выходы softmax в качестве весов. Полученный вектор и будет новым представлением исходного токена.

На рисунке 1.13 изображен пример работы self-attention.

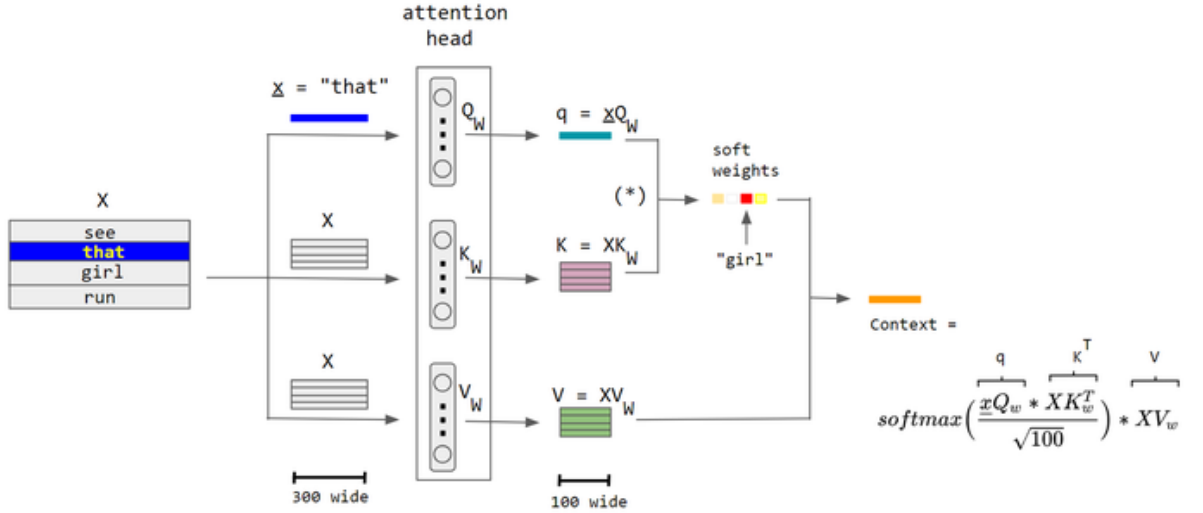


Рисунок 1.13 – Пример работы механизма внутреннего внимания

В данном примере также происходит деление на нормализующую константу $\sqrt{100}$ (квадратный корень из размерности key-векторов). Исходя из оригинальной статьи, это приводит к более стабильным градиентам в процессе обучения.

Вместо обычного self-attention чаще всего используют multi-head self-attention, в котором используется несколько блоков self-attention, результаты которых конкатенируются и умножаются на некоторую матрицу весов W_O . Это позволяет разным self-attention головам фокусироваться на разных взаимосвязях. Кроме того, разные головы могут вычисляться параллельно. В совокупности эти два плюса значительно увеличивают возможности внутреннего внимания моделировать взаимосвязи между словами.

Таким образом, механизм self-attention имеет вид:

$$head(Q, K, V) = softmax\left(\frac{QK^T}{norm_const}\right) V,$$

где $Q = W_Q * X, K = W_K * X, V = W_V * X$.

Multi-head self-attention имеет же следующий вид:

$$MultiHead(Q, K, V) = Concat(head_1, head_2, \dots, head_n)W^O$$

1.4.2 Архитектура трансформера

На рисунке 1.14 изображена архитектура трансформера из оригинальной статьи:

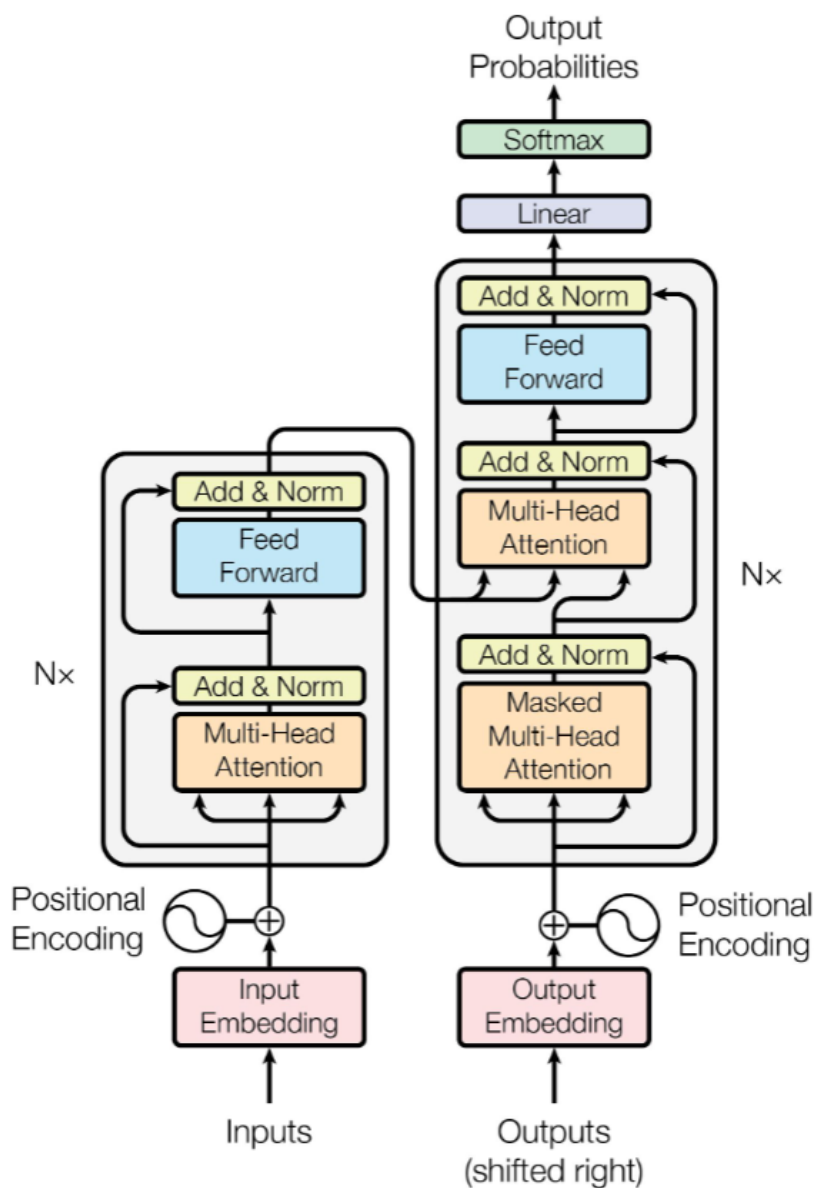


Рисунок 1.14 – Архитектура трансформера

Левую часть трансформера называют энкодером, а правую - декодером. Энкодер улучшает представление эмбеддингов токенов исходного текста, а также дает возможность декодеру с помощью multi-head attention смотреть на представления токенов исходного текста. Дело в том, что изначально данная архитектура использовалась для перевода текста, т.е. декодер отвечал за перевод (генерацию следующих токенов). Т.к. процессе перевода важно опираться на исходный текст, здесь и присутствует такой механизм.

Заметим также, что в декодере есть механизм masked multi-head

attention. Отличается он от обычного multi-head attention тем, что этот механизм внимания в декодере будет работать только с уже сгенерированными токенами.

Важно также отметить, что в трансформере также используются позиционные эмбединги для того, чтобы учитывать порядок токенов в тексте. Существуют разные методы кодирования позиций. Исторически первым подходом было использование фиксированных эмбедингов, однозначно кодирующих позицию тригонометрическими функциями [5]:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right),$$

где pos — номер позиции, i — индекс элемента в векторе, который кодирует данную позицию, d — размерность эмбединга.

С появлением трансформеров появились другие способы кодирования позиций. Например, можно сделать позиционные эмбединги обучаемыми, либо использовать относительные позиционные представления, как на рисунке 1.15.

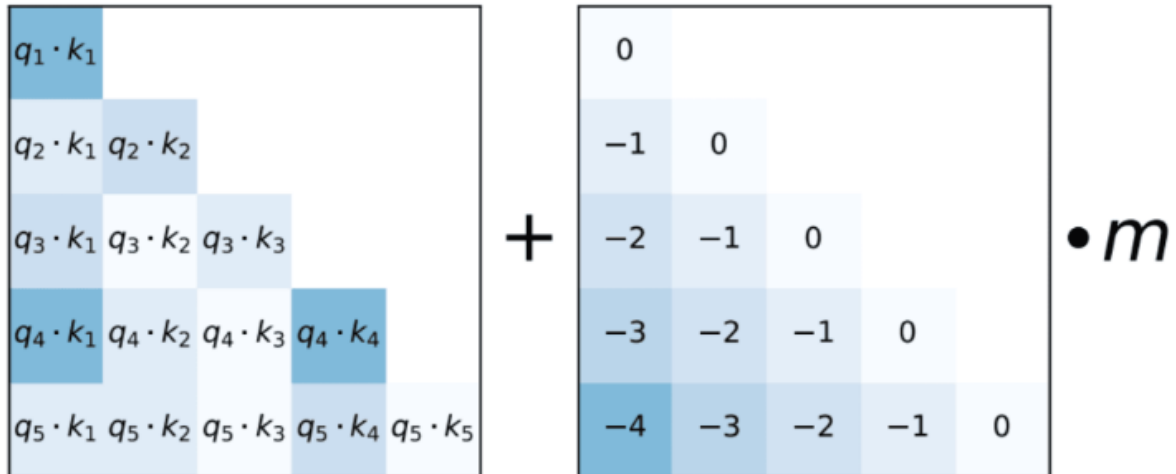


Рисунок 1.15 – Метод позиционного кодирования ALiBi

На данный момент существует множество моделей, использующих в той или иной степени архитектуру трансформера. Одними из наиболее популярных моделей являются BERT и GPT. Хотя с помощью GPT тоже можно решать задачу предсказания заработной платы, но тем не менее наиболее подходящей под нашу задачу является модель BERT и ей подобные. BERT (Bidirectional Encoder Representations from Transformers) представляет собой модель, основанную на архитектуре трансформера, предназначенную для

предобучения языковых представлений с целью их последующего применения в широком спектре задач обработки естественного языка, разработанную компанией Google. BERT обучился на огромном объеме текстовых данных корпуса BooksCorpus и англоязычной Википедии. При обучении BERT предсказывает, какие слова отсутствуют в предложении на основе контекста, который ему предоставлен. Это задача заполнения пропусков в предложении (masked language modeling). Также BERT обучался предсказывать, является ли следующее предложение продолжением предыдущего (бинарная классификация), что помогает модели лучше понимать связь между предложениями [6].

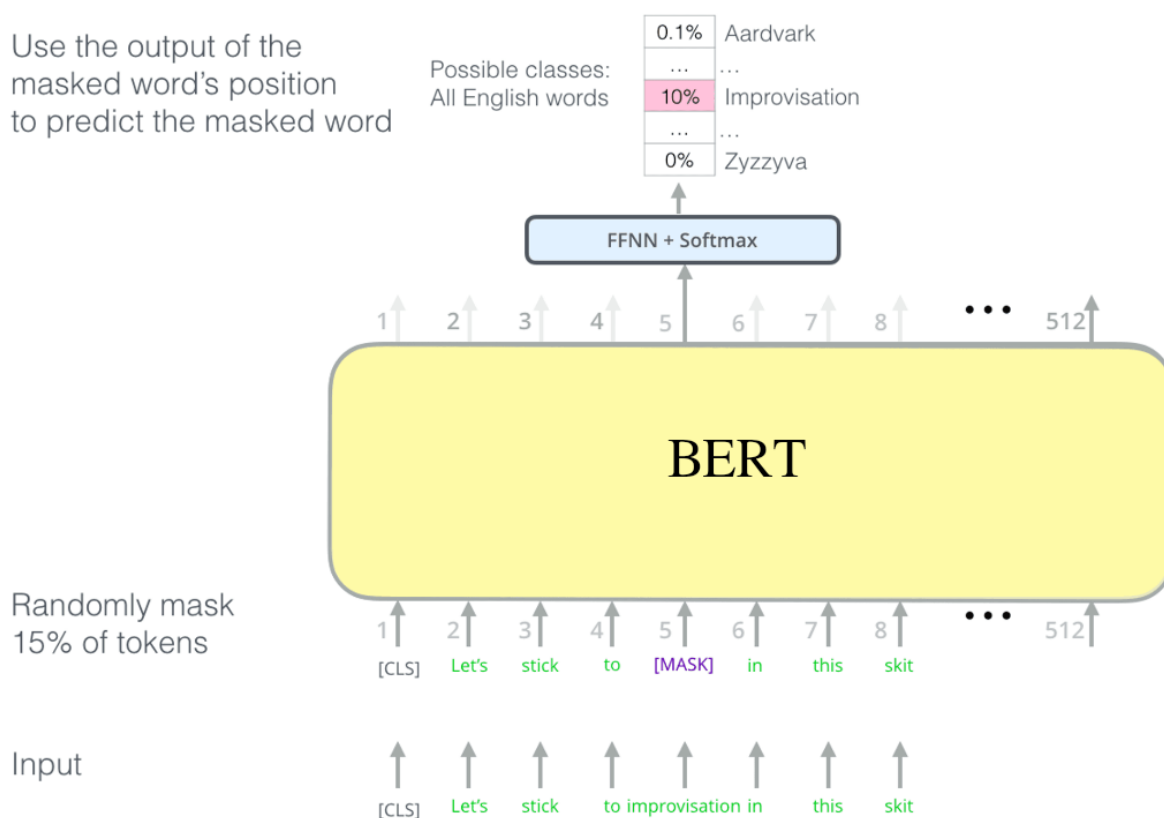


Рисунок 1.16 – Схема работы модели BERT в задаче заполнения пропусков в предложении

Т.е. BERT обучался на двух задачах. В первой, как видно на рисунке 1.16, применяется специальный токен *[MASK]*, вместо которого и предсказывается недостающее слово на основе контекста. В исходном датасете, состоящем из различных текстов интернета, применяется своего рода аугментация, которая заменяет случайное слово (например, с вероятностью 15%, как на картинке) на данный токен. Таким образом, можно сказать, что датасет увеличился во много раз. Во второй задаче, что так же видно на картинке, применяется специальный токен *[CLS]* в начале текста, полносвязный слой

над которым применяется для предсказания того, является ли следующее предложение продолжением предыдущего, причем предложения разделяются специальным токеном $[SEP]$. Таким образом, BERT позволяет получить представления слов и фраз, учитывающие их контекст и семантическую связь с другими словами в предложении. Это позволяет использовать BERT для различных задач NLP, включая классификацию текста, вопросно-ответные системы, анализ тональности и другие. BERT может быть дообучен на узкоспециализированных задачах, в том числе и на задачу регрессии (в нашем случае — задача предсказания заработной платы), используя, например, полносвязный слой над $[CLS]$ токеном (поскольку данный токен содержит по сути представление исходного текста), что делает его гибким инструментом для различных приложений в NLP.

Помимо всего вышеперечисленного, стоит также упомянуть о токенизации в трансформерах. В обычной токенизации слов существует проблема фиксированного словаря. Любые токены, которые не входят в наш фиксированный словарь, будут заменены на специальный токен $< UNK >$. Т.е. существует возможность того, что мы получим в тексте большое количество токенов $< UNK >$, из-за чего модель может потерять много информации. Эту проблему решает токенизация подслов. В такой токенизации мы сможем разбить слова на подслова, которые есть в словаре, и таким образом мы сможем закодировать все слова в тексте.

Существует несколько методов такой токенизации, рассмотрим два наиболее популярных метода: BPE (Byte-Pair-Encoding) и WordPiece. Алгоритм BPE работает следующим образом. Вначале составляется словарь, состоящий только из символов исходного алфавита, также устанавливается желаемое ограничение на размер словаря. Далее, пока не достигнуто ограничение на размер словаря, назначается новый токен, являющийся объединением двух существующих токенов из словаря, которые вместе встречаются чаще других пар токенов в корпусе, как на рисунке 1.17 [7].

Для того, чтобы отличить слова от подслов, в BPE и WordPiece используются специальные символы на префиксе подслов. Так, например, слово word может быть разбито на следующие токены: w, ##o, ##r, ##d.

В WordPiece алгоритм работает практически таким же образом, как и в BPE [8]. Единственное отличие состоит в правиле слияния. Сливаются те токены, у которых принимает максимальное значение выражение следующего вида:

$$score = \frac{freq_of_pair}{freq_of_first_element \times freq_of_second_element}$$

AABABCSABBAABAC

ADD~~C~~DBAD~~A~~C

ED~~C~~D~~B~~EAC

AA – 2

AB – 4 AB = D

BA – 3

BC – 1

CA – 1

BB – 1

AC – 1

AD – 2 AD = E

DD – 1

DC – 1

CD – 1

DB – 1

DA – 1

AC – 1

Рисунок 1.17 – Схема работы BPE токенизации

Т.е. логика та же, что и в BPE, но приоритет дается парам токенов, которые сами по себе встречаются менее часто.

На практике оба метода токенизации активно используются во многих трансформерах. Например, BPE токенизация применяется в GPT-2, RoBERTa, LLaMA, а WordPiece – во многих моделях трансформеров, основанных на BERT, таких как DistilBERT, MobileBERT.

ГЛАВА 2

РЕШЕНИЕ ИСХОДНОЙ ЗАДАЧИ

Для разных датасетов разработаем и обучим различные модели предсказания заработной платы, после чего сравним качество предсказаний. Первый датасет готовый и взят из платформы kaggle, содержит английские вакансии, собранные с сайта Adzuna [9]. Второй датасет был собран с сайта белорусских вакансий rabota.by, используя HeadHunter API [10]. Для практического применения моделей разработаем телеграм-бот, предсказывающий заработную плату вакансии по ссылке на эту вакансию с сайта rabota.by.

2.1 Датасет Adzuna

2.1.1 Предобработка данных

На рисунке 2.1 можно увидеть, как выглядят данные исходной задачи.

	Id	Title	FullDescription	LocationRaw	LocationNormalized	ContractType	ContractTime	Company	Category	SalaryRaw
0	12612628	Engineering Systems Analyst	Engineering Systems Analyst Dorking Surrey Sal...	Dorking, Surrey, Surrey	Dorking	NaN	permanent	Gregory Martin International	Engineering Jobs	20000 - 30000/annum 20-30K
1	12612830	Stress Engineer Glasgow	Stress Engineer Glasgow Salary **** to **** We...	Glasgow, Scotland, Scotland	Glasgow	NaN	permanent	Gregory Martin International	Engineering Jobs	25000 - 35000/annum 25-35K
2	12612844	Modelling and simulation analyst	Mathematical Modeller / Simulation Analyst / O...	Hampshire, South East, South East	Hampshire	NaN	permanent	Gregory Martin International	Engineering Jobs	20000 - 40000/annum 20-40K
3	12613049	Engineering Systems Analyst / Mathematical Mod...	Engineering Systems Analyst / Mathematical Mod...	Surrey, South East, South East	Surrey	NaN	permanent	Gregory Martin International	Engineering Jobs	25000 - 30000/annum 25K-30K negotiable
4	12613647	Pioneer, Miser Engineering Systems Analyst	Pioneer, Miser Engineering Systems Analyst Do...	Surrey, South East, South East	Surrey	NaN	permanent	Gregory Martin International	Engineering Jobs	20000 - 30000/annum 20-30K

Рисунок 2.1 – Датасет английских вакансий

Данные состоят из следующих колонок:

1. Id — уникальный идентификатор вакансии.
2. Title — заголовок вакансии, обычно краткое описание должности.
3. FullDescription — полный текст вакансии.
4. LocationRaw — местоположение, предоставленное работодателем.
5. LocationNormalized — местоположение.
6. ContractType — полная/неполная занятость.

7. ContractTime — контракт с истечением/без истечения срока действия.
 8. Company — имя работодателя.
 9. Category — к какой из 30 категорий относится вакансия.
 10. SalaryRaw — годовая зарплата, предоставленная работодателем.
 11. SalaryNormalized — годовая зарплата.
 12. SourceName — название вебсайта или рекламодателя.
- Всего в датасете 244768 вакансий.

Предсказывать зарплату мы будем используя MSE в качестве функции потерь:

$$MSE = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2,$$

где Y_i - таргет, а $\hat{Y}_i$ - предсказанное значение.

Рассмотрим распределение зарплат в исходных данных, изображенное на рисунке 2.2:

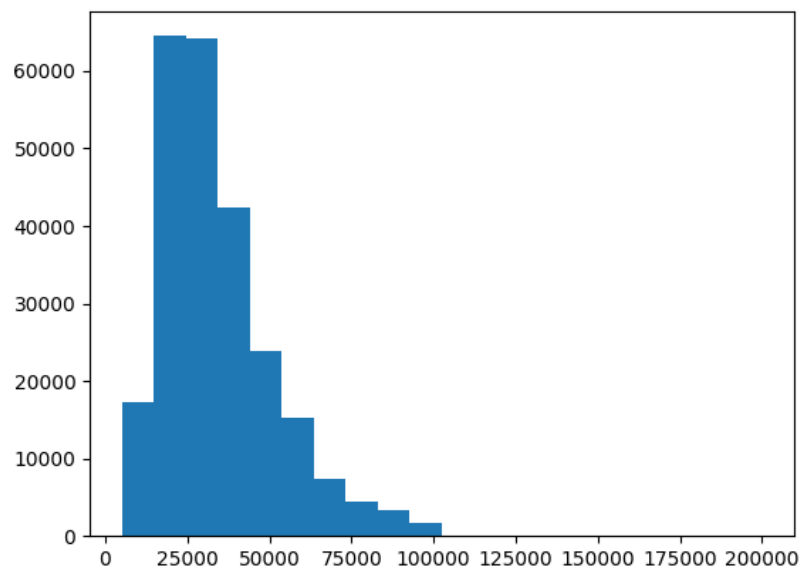


Рисунок 2.2 – Распределение зарплат английских вакансий

Т.е. мы имеем маленькое количество больших зарплат и большое количество средних зарплат. Такое распределение негативно скажется на обучении с выбранной функцией потерь. Поэтому вместо исходной зарплаты x мы будем предсказывать $\log(1 + x)$ от нее, который имеет распределение, изображенное на рисунке 2.3.

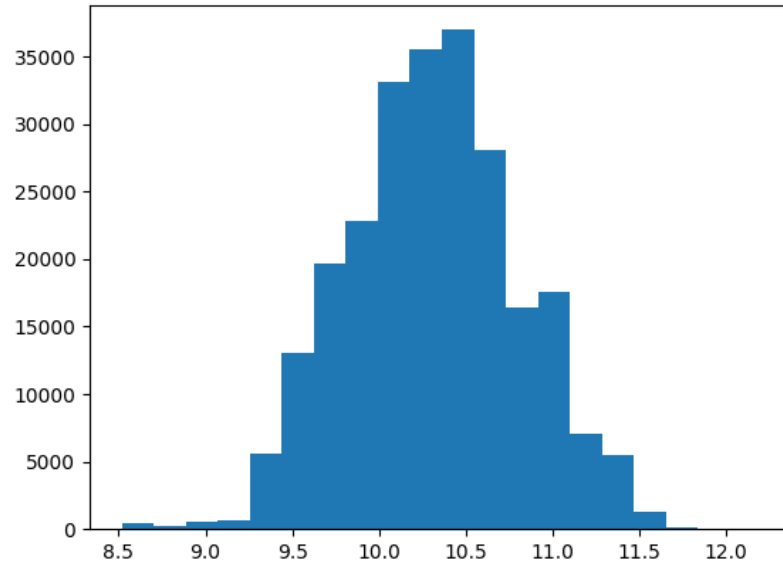


Рисунок 2.3 – Распределение $\log(1 + x)$ от зарплат английских вакансий

В качестве категориальных признаков будем использовать колонки `Category`, `Company`, `LocationNormalized`, `ContractType`, `ContractTime`. Данные категориальные признаки будут представлены в виде одного вектора, состоящего из нулей и единиц. В качестве текстовых признаков будут браться `Title` и `Description`. Из них будут извлекаться полезные признаки с помощью нейросетевых архитектур, рассмотренных выше.

В качестве токенов мы будем рассматривать как отдельные слова в тексте, так и отдельные знаки препинания. Каждому уникальному токену мы сопоставим некоторый уникальный `id`, который в дальнейшем пригодится для обучения эмбедингов. Редко встречающиеся в описаниях и названиях вакансий токены мы заменим на токен `UNK`, который может пригодиться, если на тестовых данных будут встречаться токены, которых не было на тренировочных данных. Также введем специальный токен `PAD`, который не несет никакого смысла. Он нужен для заполнения пространства, поскольку в процессе обучения важно, чтобы длина последовательностей в одном батче была одинаковой.

2.1.2 Архитектуры моделей

Исходную задачу будем решать с помощью двух разных нейросетевых моделей. Архитектура первой модели изображена на рисунке 2.4.

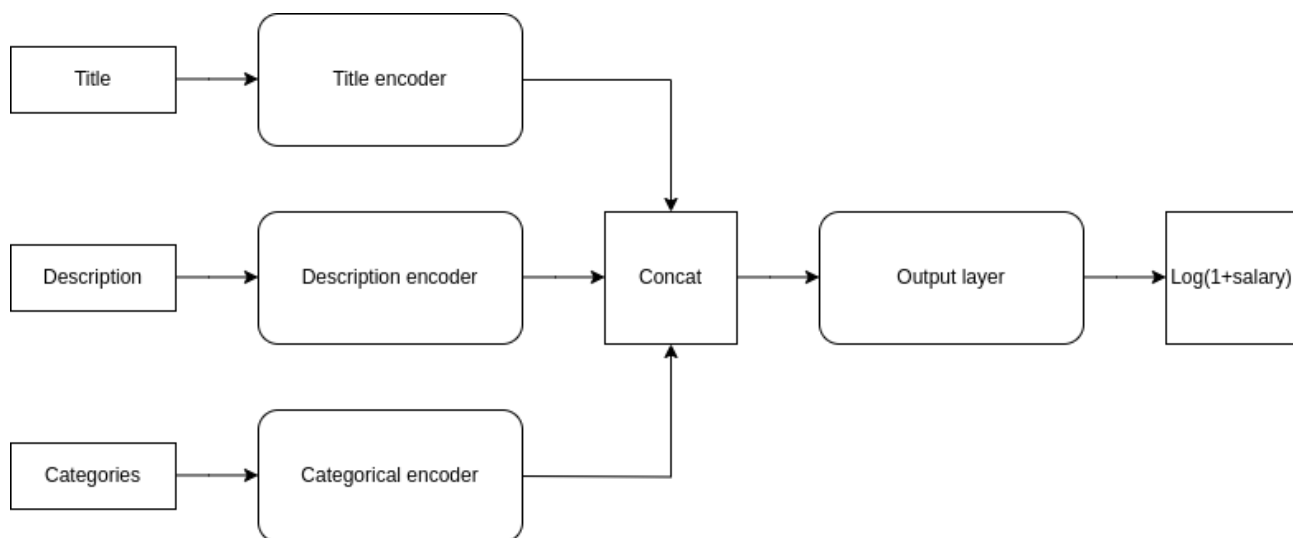


Рисунок 2.4 – Архитектура первой модели

Т.е. в модели есть три блока, отвечающие за генерацию различных признаков из вакансии. Title encoder берет информацию из названия вакансии, Description encoder из ее описания, а Categorical encoder берет информацию из категориальных признаков (Category, Company, LocationNormalized, ContractType и ContractTime). Полученные признаки из этих блоков (вектора признаков) далее конкатенируются и пропускаются через блок Output layer, который генерирует ответ: $answer = \log(1 + salary)$. Из него далее можно будет получить заработную плату:

$$salary = 1 - e^{answer}$$

В качестве Categorical encoder и Output layer мы возьмем полносвязные нейронные сети, в качестве title encoder возьмем одномерную свертку, а в качестве description encoder — двунаправленную LSTM. В качестве эмбедингов возьмем предобученные GloVe эмбединги (glove-wiki-gigaword-300, т.е. эмбединги, обученные на википедии, с количеством признаков равным 300), и будем их дообучать, чтобы они лучше подстроились под нашу задачу. В более подробном варианте архитектура изображена на рисунке 2.5.

Архитектура второй модели изображена на рисунке 2.6. Как видим, данная архитектура практически не отличается от первой. Единственное отличие — совмещенный блок для кодировщика названия и описания вакансии, которому на вход подается название и описание одним текстом, разделенные специальным символом. В качестве такого символа возьмем точку с запятой.

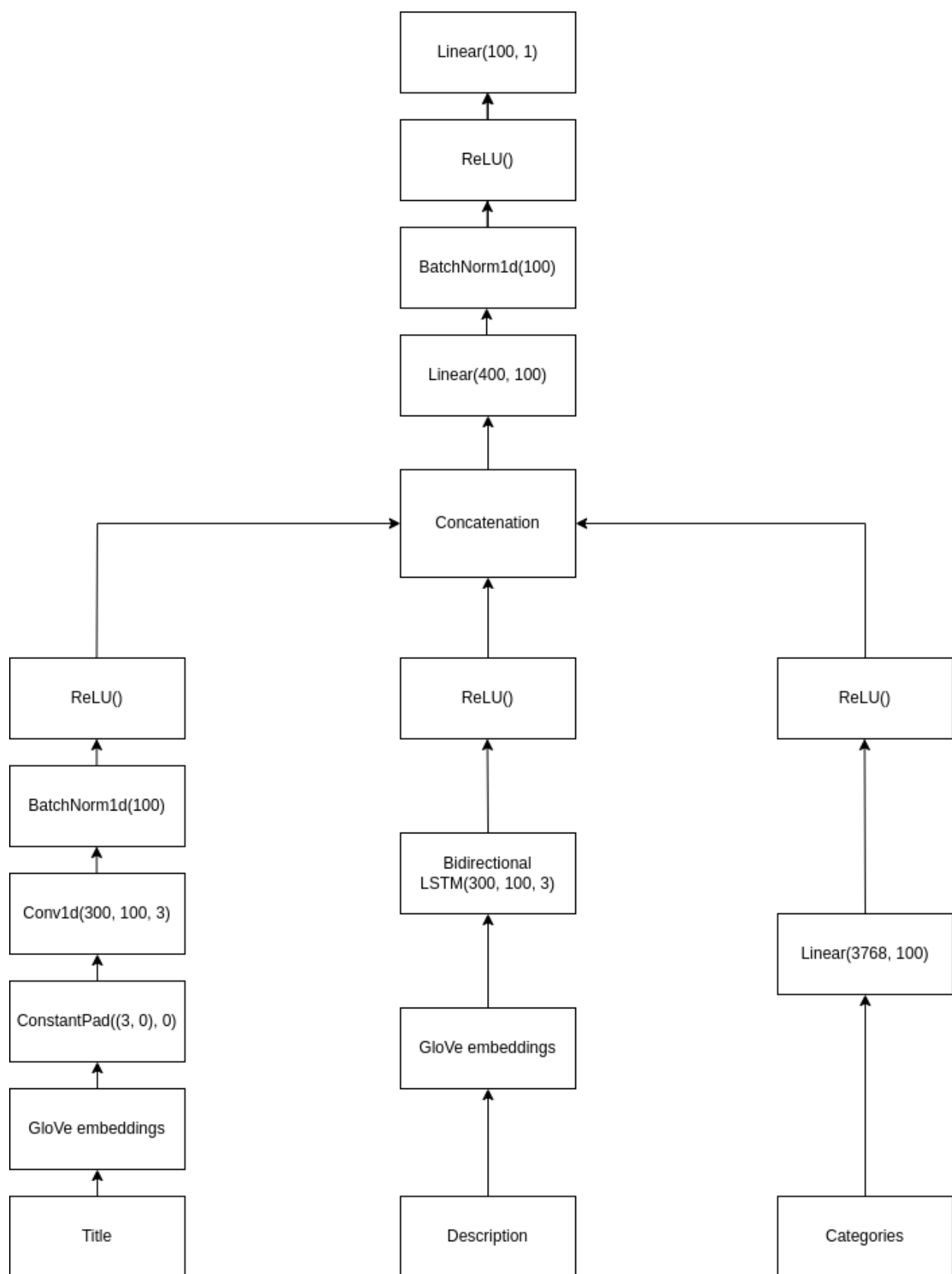


Рисунок 2.5 – Архитектура первой модели в более подробном варианте

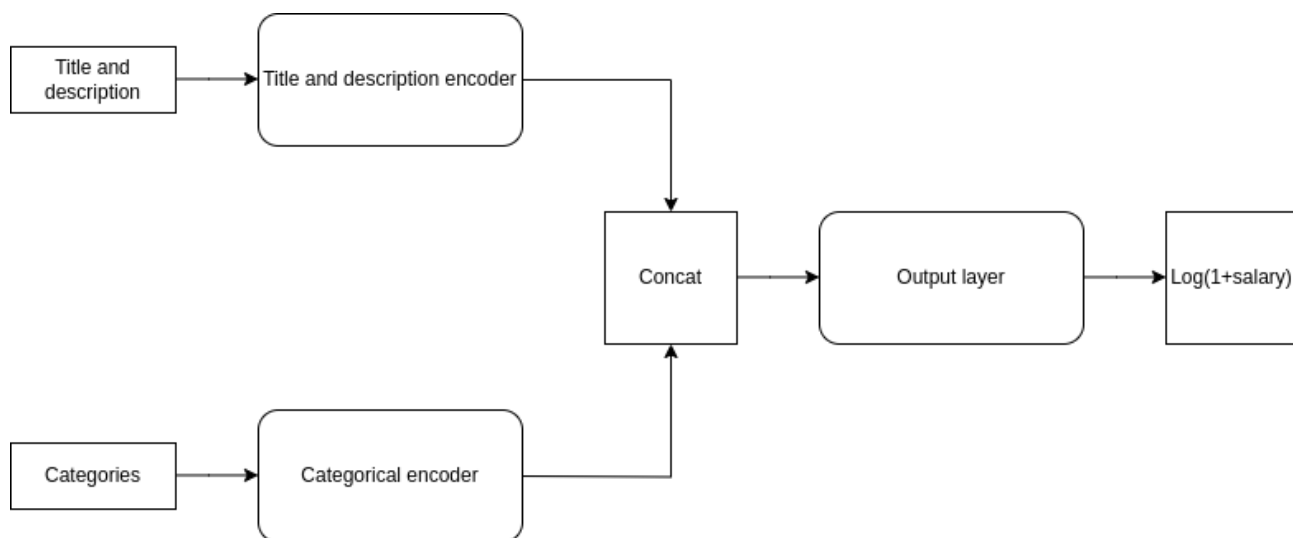


Рисунок 2.6 – Архитектура второй модели

Такое изменение в архитектуре обусловлено тем, что в данной архитектуре для получения информации из текста будет использоваться BERT, который будет дообучаться на исходной задаче. Использовать трансформер или же какую-то другую модель для получения признаков маленького названия не имеет смысла, т.к. трансформер и сам может легко обучиться понимать, что ему на вход пришли название с текстом, разделенные точкой с запятой. Конечно, использование трансформера отдельно для названия и описания вакансии не навредило бы качеству модели, однако сама модель увеличилась бы в 2 раза при том, что ее качество бы практически не улучшилось. На рисунке 2.7 данная архитектура представлена в более подробном варианте.

Т.к. обычный BERT выводит последовательность эмбеддингов токенов текста, а нам нужен вектор представления исходного текста, то для нашей задачи мы будем использовать только выход [CLS] токена, обработанный дополнительным полносвязным слоем. На изображении блок BERT с дополнительным полносвязным слоем назван как `BERTForSequenceClassification(num_labels=100)`, поскольку одноименная модель, имеющая нужную нам архитектуру, есть в библиотеке `transformers`, которую мы будем использовать для загрузки данной модели. Все остальное будет как в предыдущей архитектуре. Т.е. далее мы просто конкатенируем вектор представления текста с категориальными признаками, полученными с помощью полносвязного слоя (`Categorical esncoder`), после чего обрабатываем полученный вектор дополнительным полносвязным слоем (`Output layer`), из которого получаем ответ.

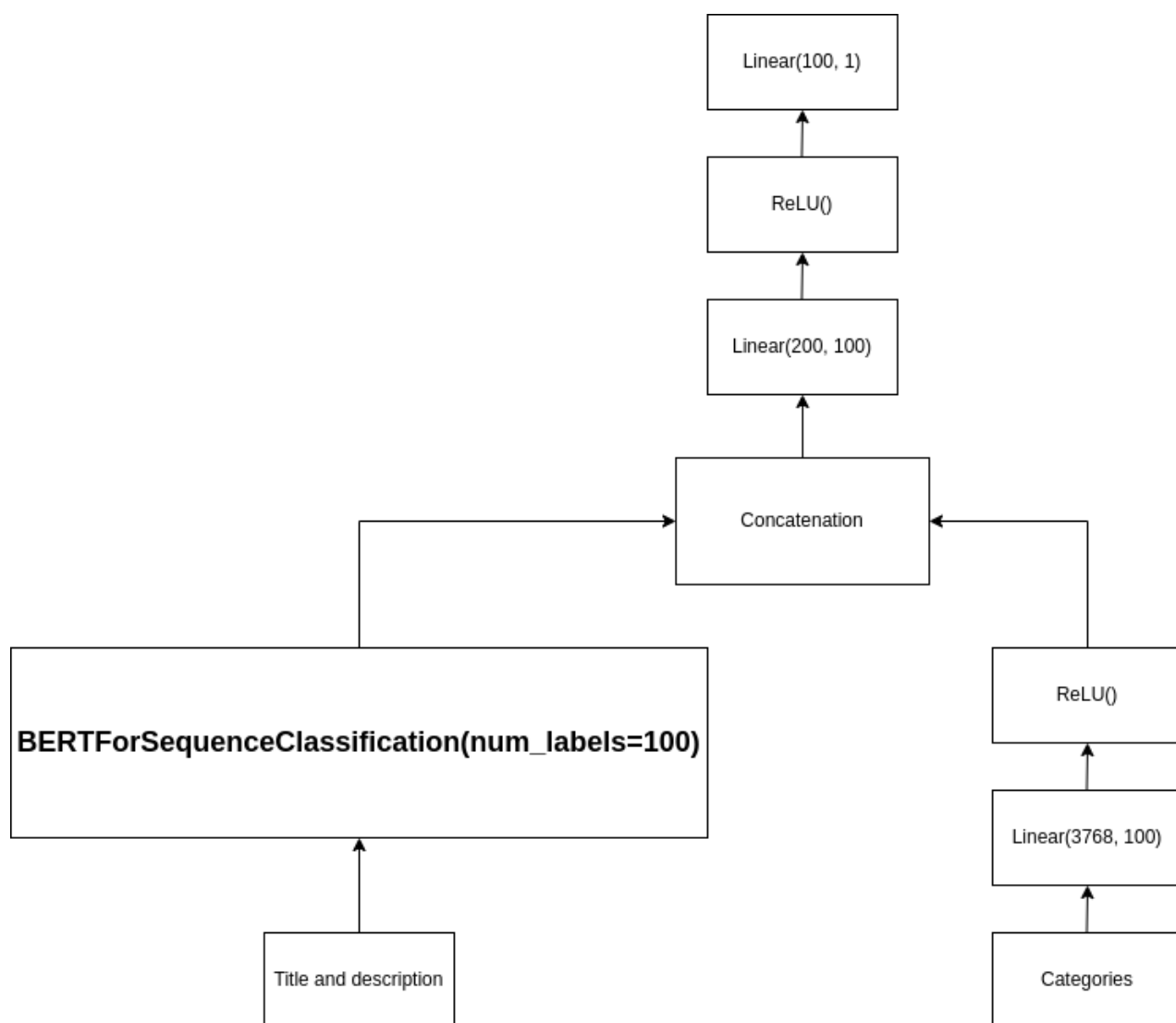


Рисунок 2.7 – Архитектура второй модели в более подробном варианте

2.1.3 Обучение моделей

Для обучения нейронных сетей воспользуемся фреймворком PyTorch языка Python, в котором есть все необходимые инструменты. В нем есть много встроенных моделей (RNN, сверточные слои, линейные слои, функции активации т.д.), оптимизаторов, lr-планировщиков и т.д. Также в PyTorch динамический граф вычислений, что облегчает написание кода для обучения. Для загрузки предобученного BERT, как уже упоминалось ранее, воспользуемся библиотекой transformers, позволяющей загружать предобученные модели с платформы huggingface. С помощью данной платформы также можно найти и загрузить предобученные GloVe эмбединги.

2.1.4 Результаты

Обучались обе модели на платформе kaggle с использованием GPU P100. Первая модель имеет 11559801 параметр, все из которых обучаемы. На рисунке 2.8 изображены графики обучения данной модели.

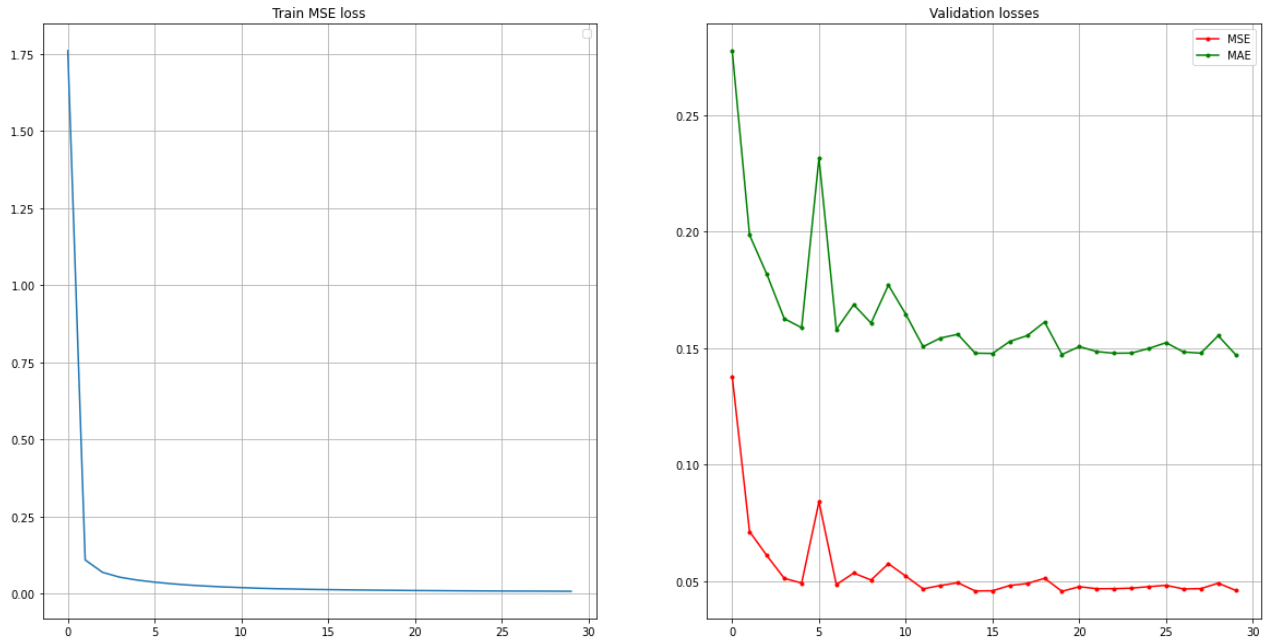


Рисунок 2.8 – Слева изображен график значений MSE первой модели после каждой эпохи на тренировочной выборке (ось абсцисс соответствует номеру эпохи, которая нумеруется с нуля, ось ординат соответствует значению MSE), справа – аналогичные графики MSE и MAE первой модели на валидационной выборке

Обучение длилось 30 эпох. Ось абсцисс на графиках соответствует номеру эпохи, ось ординат — соответствующей функции потерь. Первый график соответствует среднему MSE по батчам в конкретной эпохе на тренировочной выборке в процессе обучения. Второй график соответствует среднему MSE и MAE по батчам на валидационной выборке в конкретной эпохе (т.е. после конкретной эпохи обучения на тренировочной выборке), где MAE вычисляется по формуле:

$$MAE = \frac{1}{n} \sum_{i=1}^n |Y_i - \hat{Y}_i|$$

В качестве оптимизатора использовался Adam с $lr = 10^{-3}$, размер одного батча равнялся 32.

Во второй модели в качестве BERT был взят BERT-Tiny, он имеет гораздо меньшее число параметров, чем обычный BERT (данная модель имеет

около 4 миллионов параметров, обычный же BERT — около 110 миллионов), и его вполне хватит для нашей задачи. Итого вторая модель всего имеет 4795921 параметр, все из которых обучаемы. На рисунке 2.9 изображены графики обучения данной модели.

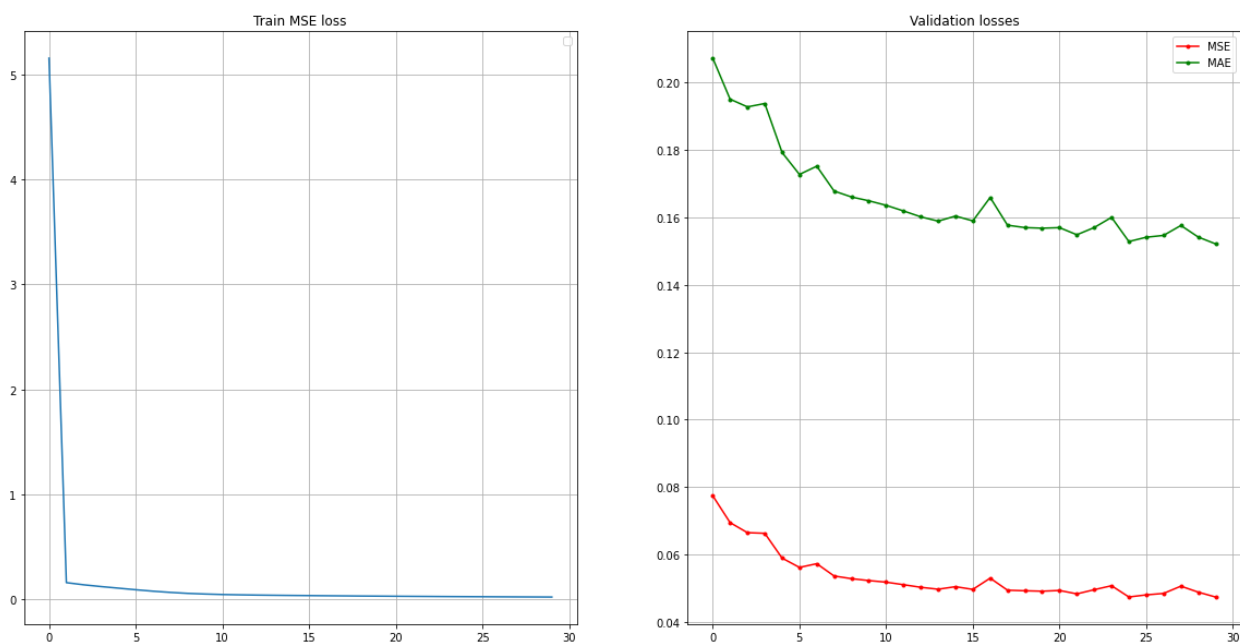


Рисунок 2.9 – Слева изображен график значений MSE второй модели после каждой эпохи на тренировочной выборке, справа – аналогичные графики MSE и MAE второй модели на валидационной выборке

Обучение также длилось 30 эпох. Смысл данных графиков аналогичный. В качестве оптимизатора аналогично использовался Adam с $lr = 2 \cdot 10^{-5}$. Использовался практически тот же загрузчик батчей, что и в первой модели. Единственное отличие — использование другого токенизатора (и, соответственно, эмбеддингов) под модель BERT-Tiny, загруженного также с платформы huggingface с помощью библиотеки transformers. Соответственно, размер батча такой же, как и для первой модели, и равен 32.

В таблице 2.1 можно увидеть итоговые результаты двух моделей.

Таблица 2.1 – Таблица с итоговыми результатами для датасета с английскими вакансиями

Model	MSE train	MSE validation	MAE validation
Conv1d + LSTM + GloVe	0.00810	0.04594	0.14721
BERT	0.02325	0.047496	0.15205

Как видим, результаты первой модели совсем немного, но лучше. Однако стоит обратить внимание, что количество параметров в первой модели

более чем в 2 раза превосходит количество параметров второй модели. Более того, из ошибки на тренировочной выборке (MSE train) и из того, насколько эта ошибка отличается от той же ошибки на валидационной выборке, можно видеть, что вторая модель менее склонна к переобучению, а значит лучше подходит для данной задачи.

2.2 Датасет rabota.by

2.2.1 Сбор данных

Сбор данных о вакансиях на сайте rabota.by осуществлялся с помощью HeadHunter API. С его помощью можно собирать информацию об интересующих вакансиях с сайтов hh.ru, rabota.by, hh1.az, hh.uz, hh.kz, headhunter.ge и headhunter.kg. К примеру, для просмотра данных об определенной вакансии можно воспользоваться запросом, который изображен на рисунке 2.10.

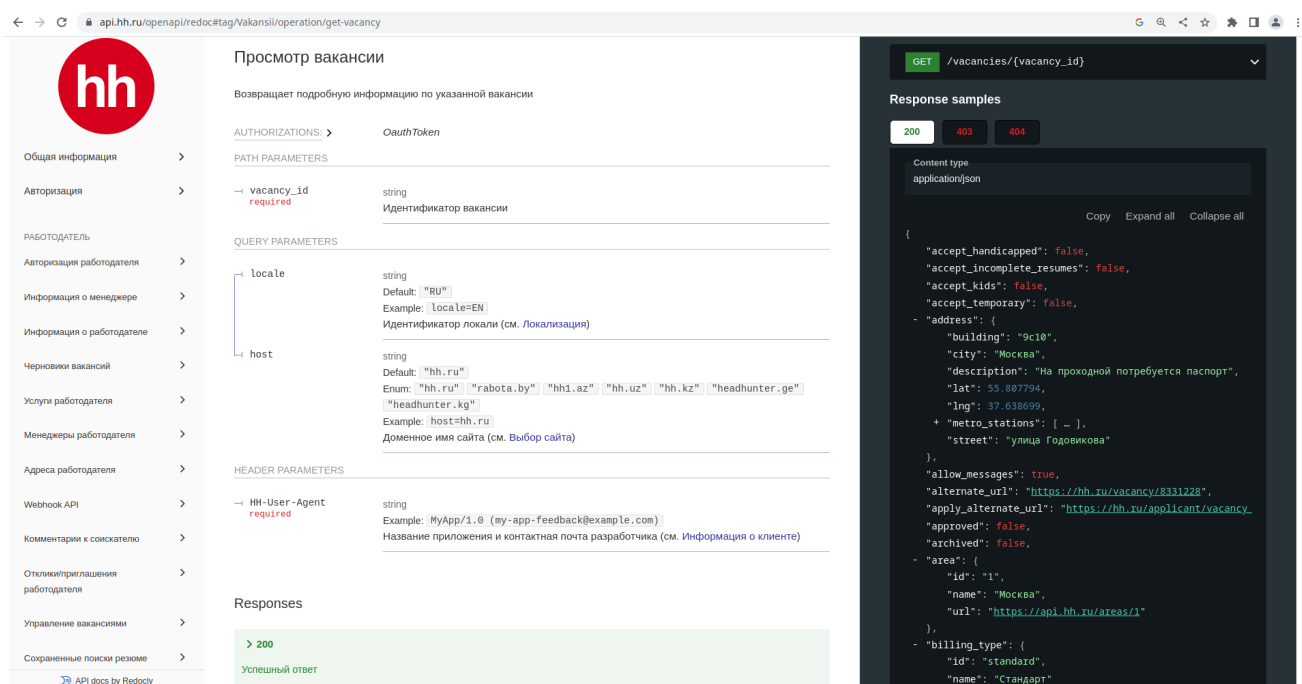


Рисунок 2.10 – HeadHunter API для получения данных об определенной вакансии

К сожалению, HeadHunter API позволяет собрать только данные о вакансиях, опубликованных за последний месяц. Для того, чтобы добиться максимального качества, были собраны данные обо всех вакансиях, опубликованных за последний месяц. Всего получилось собрать 29692 вакансии. Данные о вакансиях, выдаваемых HeadHunter API, хранились в формате json, для

удобства они были сохранены в csv формате.

2.2.2 Предобработка данных

На рисунке 2.11 можно увидеть, как выглядят собранные данные.

	id	premium	billing_type	relations	name	insider_interview	response_letter_required	area	salary	type
0	95985328	False	{'id': 'standard', 'name': 'Стандарт'}	□	Подсобный рабочий	None	False	{'id': '1002', 'name': 'Минск', 'url': 'https://...'	None	{'id': 'open', 'name': 'Открытая'}
1	94218916	False	{'id': 'standard', 'name': 'Стандарт'}	□	Старший контролер-кассир	None	False	{'id': '2352', 'name': 'Шклов', 'url': 'https://...'	{'from': 1100, 'to': 1250, 'currency': 'BYR', ...}	{'id': 'open', 'name': 'Открытая'}
2	97651501	False	{'id': 'standard', 'name': 'Стандарт'}	□	Каменщик	None	False	{'id': '2264', 'name': 'Орша', 'url': 'https://...'	{'from': 3000, 'to': 6000, 'currency': 'BYR', ...}	{'id': 'open', 'name': 'Открытая'}
3	98133457	False	{'id': 'standard', 'name': 'Стандарт'}	□	Водитель категории В,С в г. Минск	None	False	{'id': '1002', 'name': 'Минск', 'url': 'https://...'	{'from': 2100, 'to': None, 'currency': 'BYR', ...}	{'id': 'open', 'name': 'Открытая'}
4	98224341	False	{'id': 'standard', 'name': 'Стандарт'}	□	Продавец	None	False	{'id': '1006', 'name': 'Гродно', 'url': 'https://...'	{'from': 1000, 'to': None, 'currency': 'BYR', ...}	{'id': 'open', 'name': 'Открытая'}

Рисунок 2.11 – Данные о белорусских вакансиях до обработки

Строка соответствует определенной вакансии, колонки характеризуют определенный признак вакансии. В сырых данных всего 48 колонок, большинство из которых для обучения модели нам не понадобятся. Оставим из них только необходимые 18 колонок:

1. id — уникальный идентификатор вакансии.
2. billing_type — биллинговый тип вакансии (тип публикации вакансии).
3. name — название вакансии.
4. response_letter_required — обязательно ли заполнять сообщение при отклике на вакансию.
5. area — регион.
6. salary — заработная плата.
7. allow_messages — возможность переписки с кандидатами по данной вакансии.
8. experience — требуемый опыт работы.
9. schedule — график работы.

10. `employment` — тип занятости.
11. `description` — описание вакансии.
12. `accept_handicapped` — доступна ли вакансия для соискателей с инвалидностью.
13. `accept_kids` — доступна ли вакансия для соискателей старше 14 лет.
14. `driver_license_types` — список требуемых категорий водительских прав.
15. `accept_incomplete_resumes` — разрешен ли отклик на вакансию с неполным резюме.
16. `employer` — название компании.
17. `has_test` — наличие прикрепленного тестового задания к вакансии.
18. `accept_temporary` — доступна ли вакансия с временным трудоустройством.

Перед тем, как получить данные колонки именно с такими значениями, требуется предварительная обработка. Так, например, в `employer` изначально хранился словарь, содержащий более подробную информацию о компании, которая нам не понадобится (например, `id` компании, `url`, на который нужно сделать GET-запрос, чтобы получить информацию о компании, и т.д.), в `description` хранилось описание в формате `html` вместо обычного текста и т.д.

Также заметим, что в сырых данных зарплата указана с нижней и верхней границами. Для обучения модели нам потребуется удалить вакансии, которые не содержат заработную плату, либо содержат ее не в белорусских рублях. Поэтому, чтобы не потерять много данных, мы будем предсказывать нижнюю границу зарплаты, поскольку вакансий с указанными нижними границами зарплаты гораздо больше, чем с указанными верхними границами (примерно в 2 раза больше). Т.е. в качестве `salary` мы теперь имеем нижнюю границу зарплаты вакансии. В итоге после такой чистки данных остается 22287 вакансий. На рисунке 2.12 изображены данные после чистки.

	<code>id</code>	<code>billing_type</code>	<code>name</code>	<code>response_letter_required</code>	<code>area</code>	<code>salary</code>	<code>allow_messages</code>	<code>experience</code>	<code>schedule</code>	<code>employment</code>
1	94218916	Стандарт	Старший контролер-кассир	False	Шклов	1100	True	Нет опыта	Полный день	Полная занятость
2	97651501	Стандарт	Каменщик	False	Орша	3000	True	От 1 года до 3 лет	Полный день	Полная занятость
3	98133457	Стандарт	Водитель категории В,С в г. Минск	False	Минск	2100	True	От 1 года до 3 лет	Полный день	Полная занятость
4	98224341	Стандарт	Продавец	False	Гродно	1000	True	От 1 года до 3 лет	Полный день	Полная занятость
5	98449416	Стандарт	Инженер ПТО	False	Минск	3500	True	От 1 года до 3 лет	Полный день	Полная занятость

Рисунок 2.12 – Данные о белорусских вакансиях после обработки

Функцию потерь и метрики будем использовать такие же, как и в предыдущем датасете.

На рисунке 2.13 изображено распределение зарплат в исходных данных. Так же, как и в предыдущем датасете, оно смещено влево.

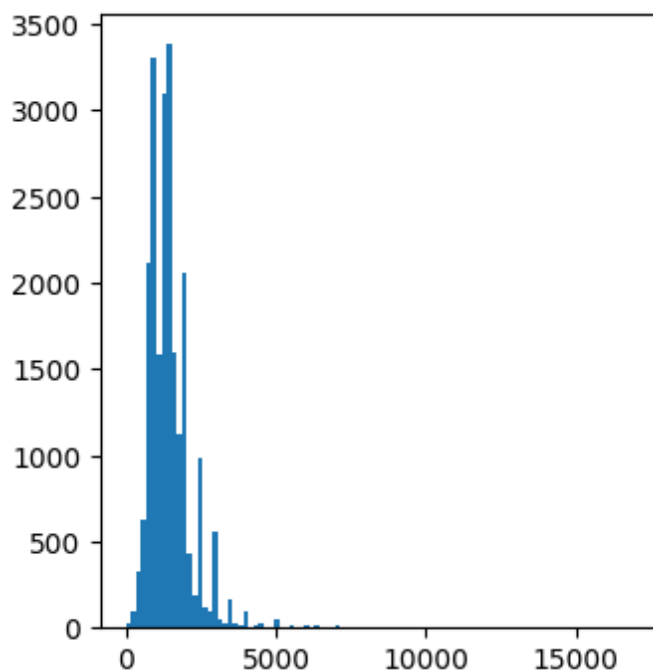


Рисунок 2.13 – Распределение зарплат белорусских вакансий

По той же причине, что и в прошлый раз, вместо исходной зарплаты x мы будем предсказывать $\log(1 + x)$ от нее, который имеет распределение, изображенное на рисунке 2.14.

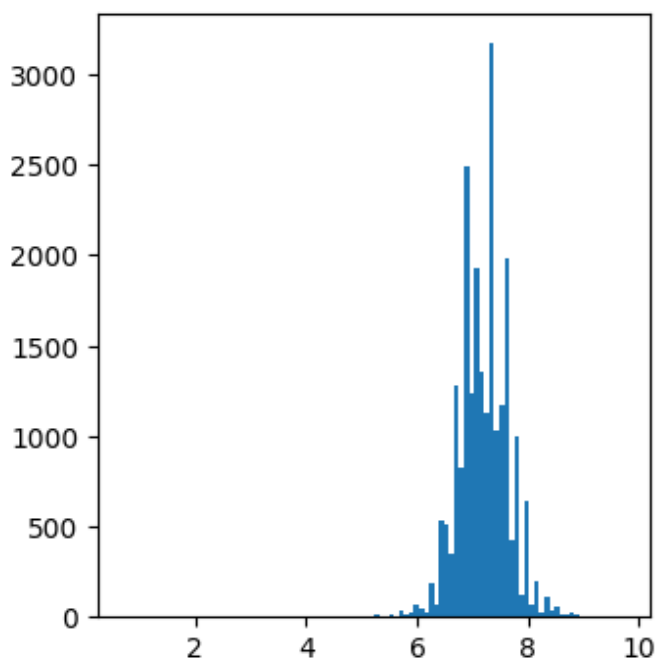


Рисунок 2.14 – Распределение $\log(1 + x)$ от зарплат белорусских вакансий

В качестве категориальных признаков будем использовать все колонки, кроме `id`, `name`, `description` и `driver_license_types`. Как и в предыдущем датасете, данные категориальные признаки будут представлены в виде одного вектора, состоящего из нулей и единиц. Признак `driver_license_types` по сути является категориальным, и можно было бы его использовать как категориальный, однако, он нужен только для вакансий, связанных с вождением, а итоговое количество признаков возросло бы примерно в 2 раза, поэтому мы просто добавим в самом конце описания вакансии с непустыми значениями данного признака требования к категориям водительских прав. Соответственно, `name` и `description` — текстовые признаки, которые будут обрабатываться специальными нейросетевыми архитектурами для работы с текстом. Токенизация такая же, как была, только в этот раз токенами являются в основном русские слова, в редких случаях английские (в том числе специальные токены UNK и PAD), и знаки препинания.

2.2.3 Архитектуры моделей

Архитектуры моделей абсолютно аналогичны предыдущим, за исключением того, что т.к. категориальных признаков в данном датасете у нас меньше, чем в прошлом (1206 вместо 3768), то, соответственно, размерность входного канала в линейном слое категориального кодировщика будет равняться 1206 вместо 3768. На этот раз проверим качество трех моделей. У первой модели архитектура аналогична архитектуре первой модели для предыдущего датасета, вторая и третья модели по архитектуре аналогичны второй модели для предыдущего датасета, но используются разные кодировщики текста. Для второй модели будем использовать `rubert-tiny` для классификации текста с `num_labels=100`. Данная модель — это по сути BERT для мультязычных текстов, дообученный на русских текстах. Для третьей модели будем использовать `rubert-base-cased` для классификации текста с `num_labels=512`. Данная модель имеет по сути ту же архитектуру, что и вторая, но имеет гораздо большее число параметров.

2.2.4 Обучение моделей

Для обучения воспользуемся теми же инструментами, что и в прошлый раз, но учитывая, что у нас теперь русскоязычный текст.

2.2.5 Результаты

Обучались обе модели так же на платформе kaggle с использованием GPU P100. Первая модель имеет 5507301 параметр, все из которых обучаемы. На рисунке 2.15 изображены графики обучения данной модели.

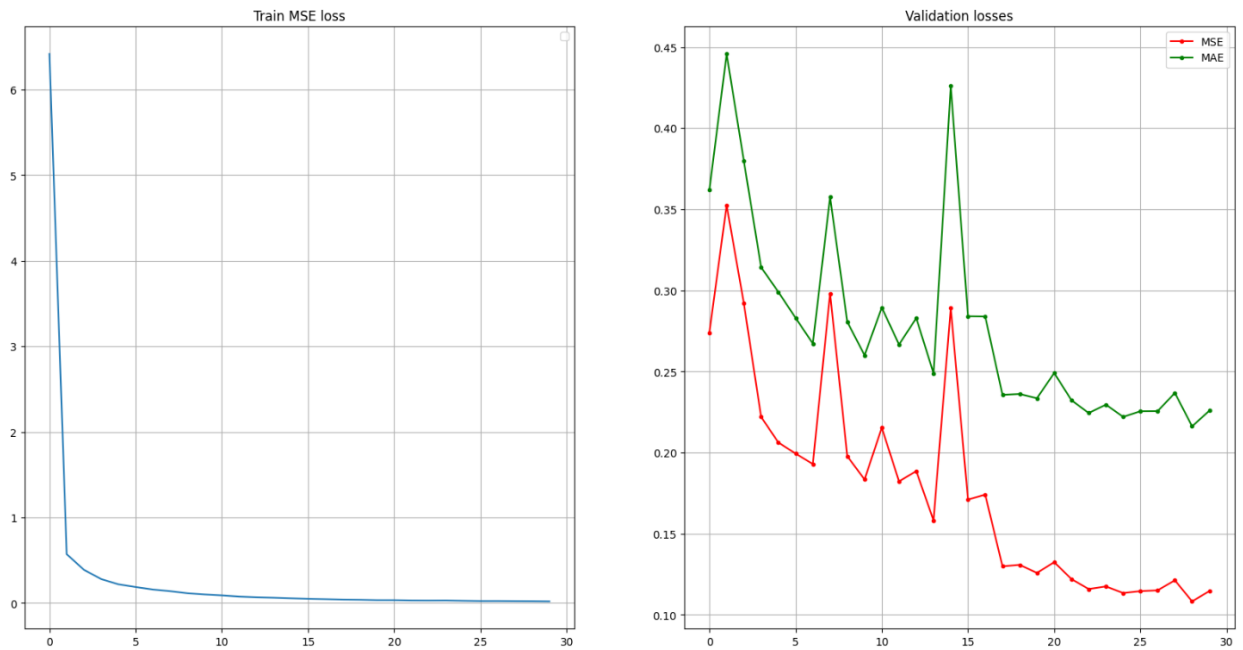


Рисунок 2.15 – Слева изображен график значений MSE первой модели после каждой эпохи на тренировочной выборке, справа – аналогичные графики MSE и MAE первой модели на валидационной выборке

Обучение длилось 30 эпох. В качестве оптимизатора использовался Adam с $lr = 10^{-3}$, размер одного батча равнялся 32.

Вторая модель, использующая rubert-tiny в качестве кодировщика текста, имеет 11815468 параметров, все их которых обучаемы. На рисунке 2.16 изображены графики обучения данной модели.

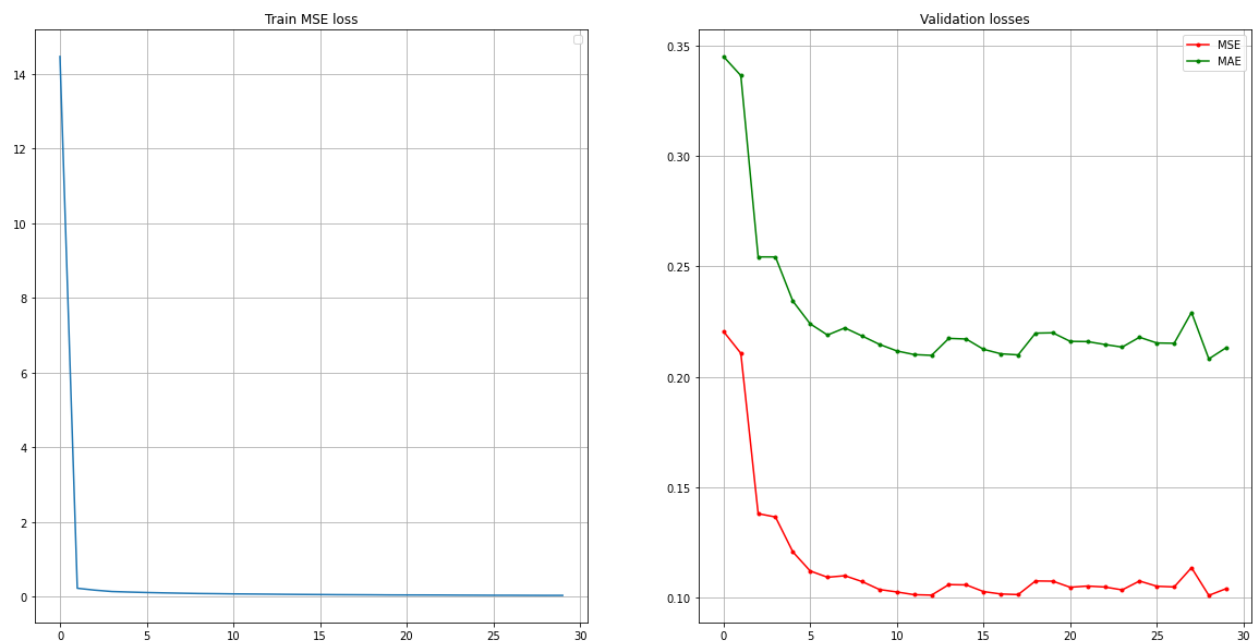


Рисунок 2.16 – Слева изображен график значений MSE второй модели после каждой эпохи на тренировочной выборке, справа – аналогичные графики MSE и MAE второй модели на валидационной выборке

Обучение длилось 30 эпох. В качестве оптимизатора аналогично использовался Adam с $lr = 2 \cdot 10^{-5}$, размер батча равнялся 32.

Третья модель, использующая rubert-base-cased в качестве кодировщика текста, имеет уже гораздо большее число параметров, в отличие от предыдущих моделей — 179390465 параметров, все из которых обучаемы. На рисунке 2.17 изображены графики обучения данной модели.

Обучение длилось 55 эпох, т.к. модель сходилась дольше. В качестве оптимизатора аналогично использовался Adam с $lr = 2 \cdot 10^{-5}$, размер батча равнялся 32.

В таблице 2.2 можно увидеть итоговые результаты трех моделей. Результаты вполне ожидаемы. Как мы убедились на примере прошлого датасета, первая модель уступает трансформерам. Поскольку в тот раз первая модель имела большее количество параметров, выгода в использовании трансформеров не была так заметна. Однако сейчас, когда ситуация поменялась в противоположную сторону, эта выгода уже более заметна. Третья модель заметно лучше остальных показала себя на валидационной выборке, что неудивительно, поскольку она имеет гораздо большее количество параметров. У данной модели есть и свои недостатки, а именно медленная сходимость, медленное обучение и инференс.

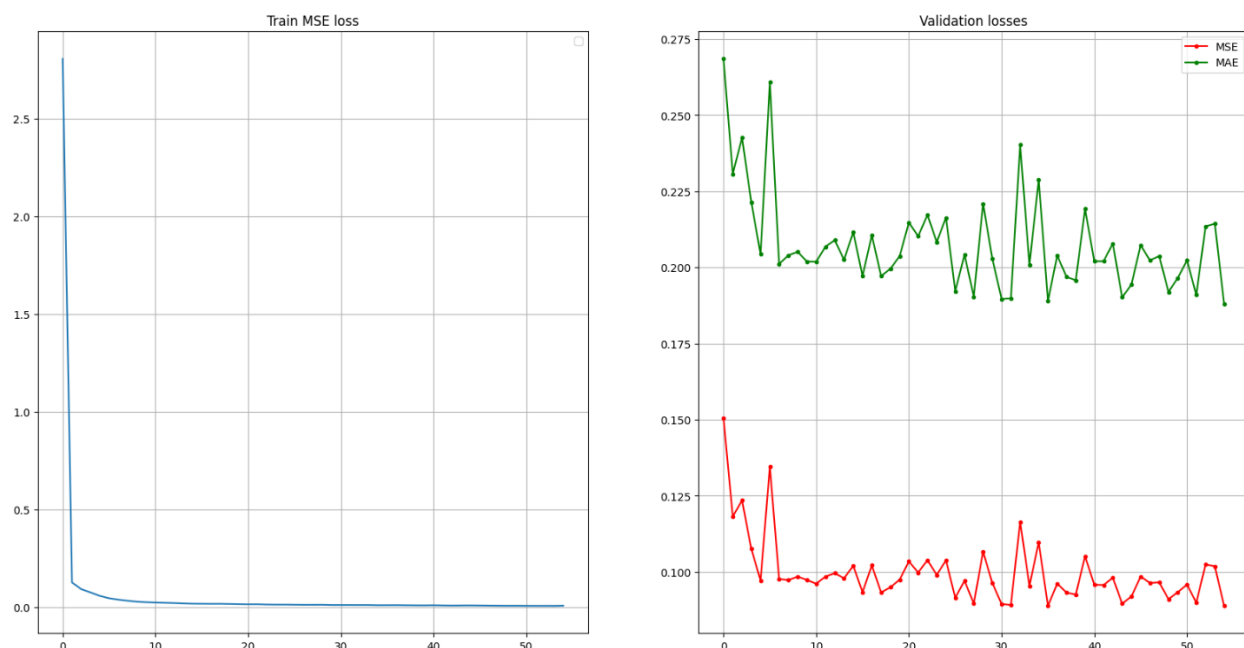


Рисунок 2.17 – Слева изображен график значений MSE третьей модели после каждой эпохи на тренировочной выборке, справа – аналогичные графики MSE и MAE третьей модели на валидационной выборке

Таблица 2.2 – Таблица с итоговыми результатами для датасета с белорусскими вакансиями

Model	MSE train	MSE validation	MAE validation
Conv1d + LSTM + GloVe	0.02030	0.11462	0.22581
rubert-tiny	0.03707	0.10419	0.21319
rubert-base-cased	0.00845	0.08887	0.18803

Отметим также, что результаты у всех моделей получились хуже, чем у моделей, обученных на датасете с английскими вакансиями, что неудивительно, поскольку датасет с белорусскими вакансиями значительно меньше датасета с английскими вакансиями.

2.2.6 Разработка телеграм-бота

Для свободного применения моделей на практике был разработан телеграм-бот, с помощью которого пользователь может получить предсказание нижней оценки заработной платы определенной вакансии с сайта rabota.by. Для этого ему нужно лишь вставить ссылку на эту вакансию в качестве аргумента специальной команды `/salary`. Кроме того, пользователю доступно и несколько других команд, позволяющих ему переключаться меж-

ду моделями, хранить историю предсказаний и т.д. Примеры взаимодействия с ботом изображены на рисунках 2.18, 2.19, 2.20, 2.21, 2.22.

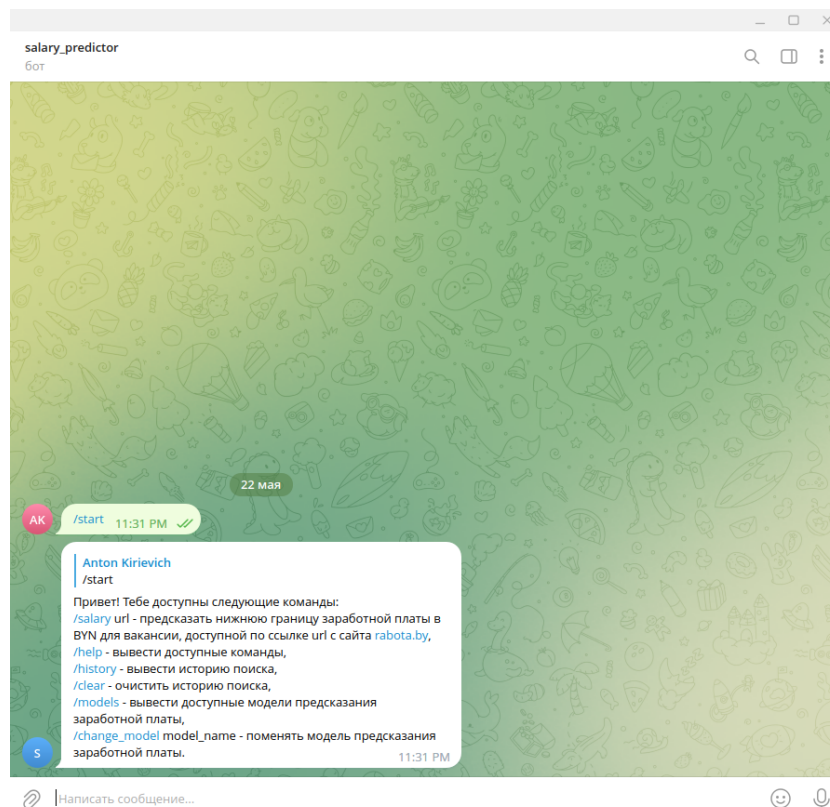


Рисунок 2.18 – Начало работы с ботом

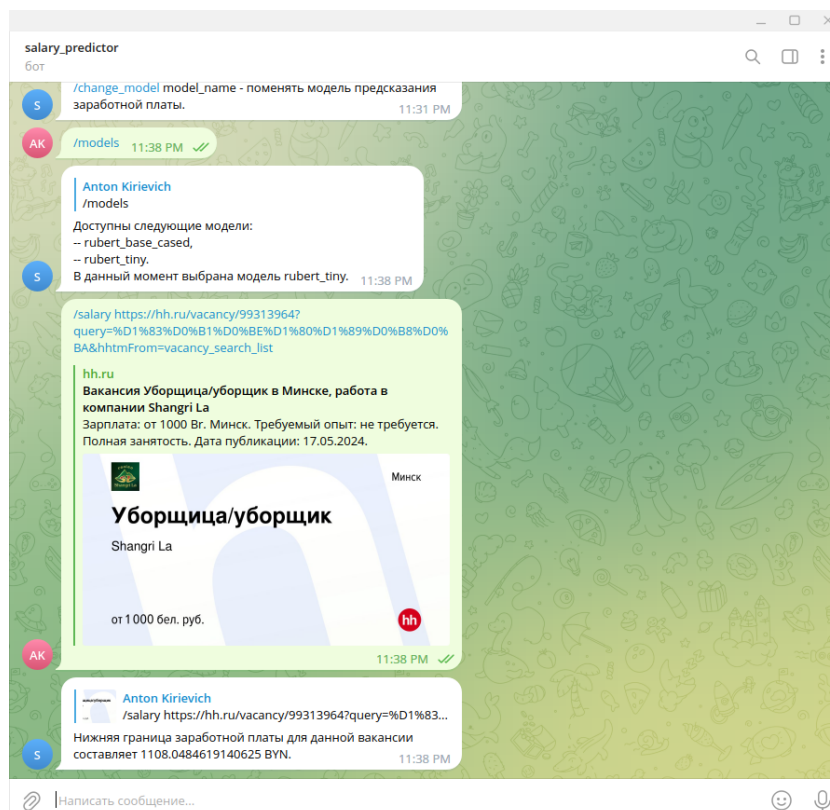


Рисунок 2.19 – Предсказание зарплаты с помощью модели rubert-tiny

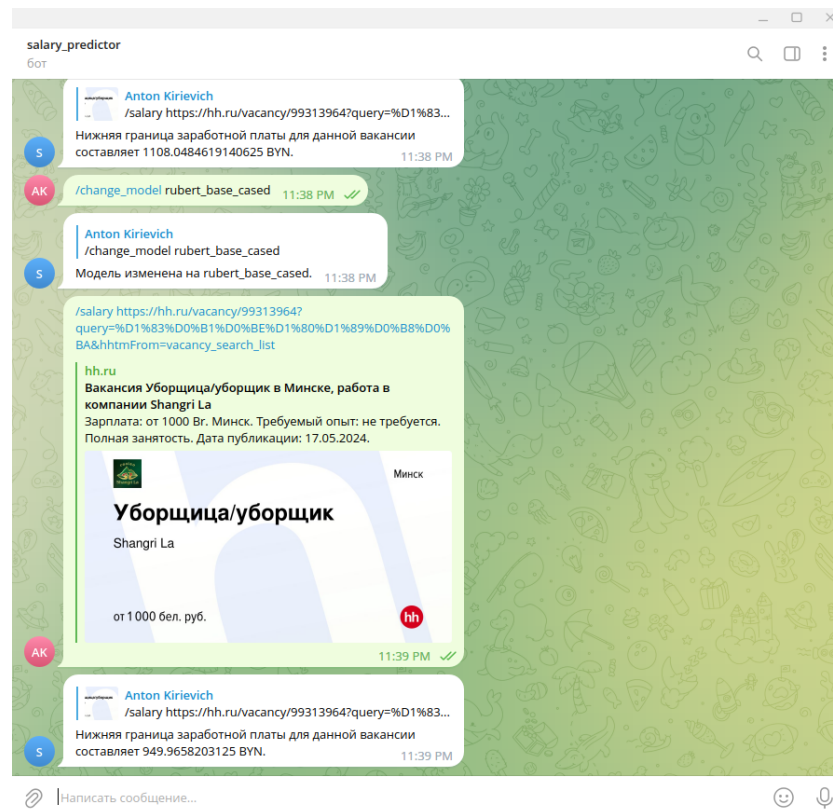


Рисунок 2.20 – Предсказание зарплаты с помощью модели rubert-base-cased

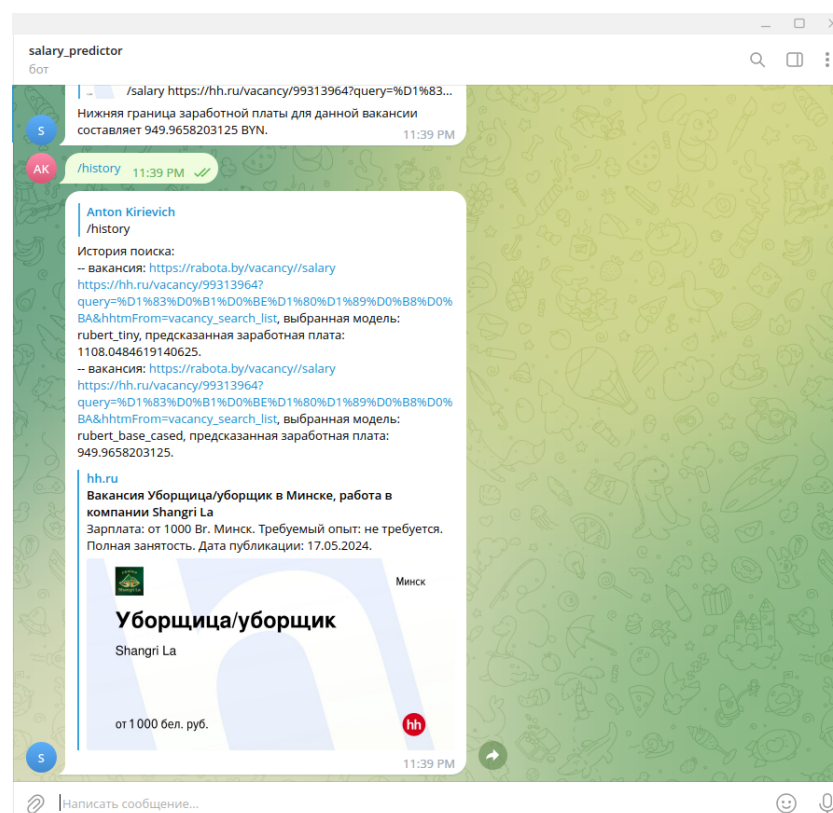


Рисунок 2.21 – Вывод истории поиска

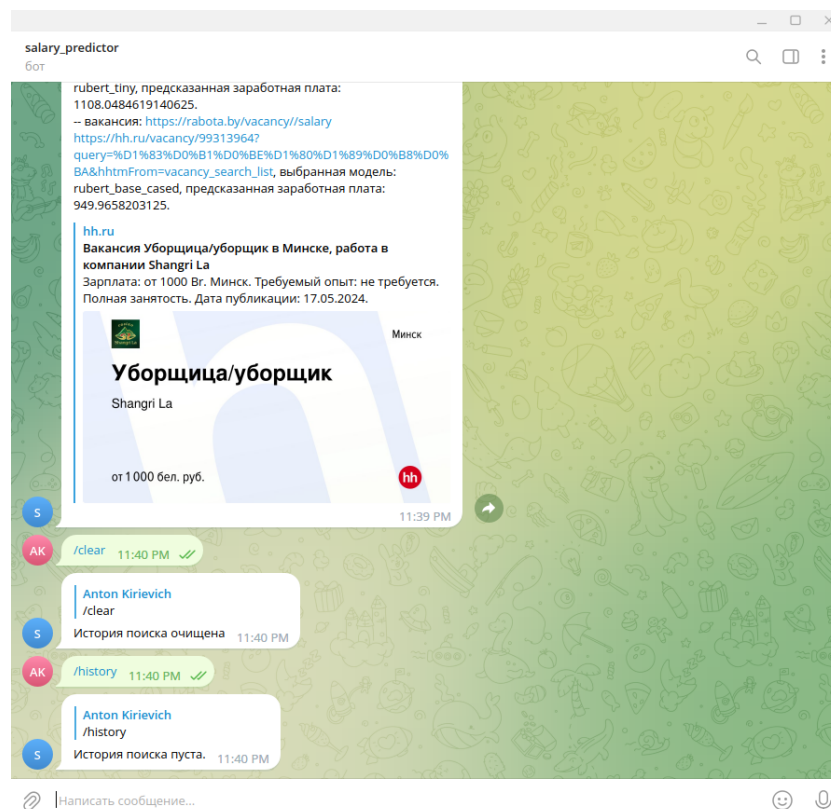


Рисунок 2.22 – Очистка истории поиска

Телеграм-бот был разработан с помощью асинхронного фреймворка aiogram языка Python.

ЗАКЛЮЧЕНИЕ

В ходе работы:

1. Представлены схема работы, основные блоки и процесс обучения полносвязных нейронных сетей, которые используются в качестве кодировщиков категориальных признаков и как выходные слои итоговых моделей.
2. Рассмотрены сверточные нейронные сети и их частный случай – одномерные сверточные нейронные сети, которые используются для кодирования текста в некоторых моделях.
3. Представлены различные методы получения эмбеддингов слов и подслов, которые используются во всех моделях, кодирующих текст.
4. Представлена схема работы рекуррентных нейронных сетей и их различные модификации, которые используются для кодирования текста в некоторых моделях.
5. Рассмотрен принцип работы multi-head self-attention, который является одним из основных блоков архитектуры трансформера.
5. Рассмотрена архитектура трансформера и предобученная модель BERT, которая использует данную архитектуру и является одной из наиболее современных моделей для кодирования исходного текста.
6. Для решения исходной задачи на готовом датасете платформы kaggle с английскими вакансиями были построены несколько архитектур, использующих в качестве отдельных блоков кодировщики для текста и категориальных признаков, которые в свою очередь используют нейросетевые архитектуры, рассмотренные ранее.
7. На основе построенных архитектур на датасете с английскими вакансиями были обучены модели предсказания заработной платы.
8. Проведено сравнение результатов, полученных моделями на датасете с английскими вакансиями.
9. Собраны данные о белорусских вакансиях с помощью HeadHunter API с сайта rabota.by.
10. Проведена очистка полученных данных о белорусских вакансиях для получения датасета в более пригодном виде для обучения.
11. На очищенных данных датасета белорусских вакансий аналогичным образом, как и на датасете с английскими вакансиями, были обучены модели с различными архитектурами.
12. Для разных моделей и датасетов было проведено сравнение полученных результатов, сделаны определенные выводы.

13. Разработан телеграм-бот, с помощью которого можно предсказать заработную плату вакансии по соответствующей ссылке с сайта rabota.by, для удобного использования полученных моделей на практике.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Первое знакомство с полносвязными нейросетями [Electronic resource]. - Mode of access: <https://education.yandex.ru/handbook/ml/article/pervoe-znakomstvo-s-polnosvyaznymi-nejrosetyami> - Date of access: 10.05.2024.
2. Convolutions and Backpropagations [Electronic resource]. - Mode of access: <https://pavisj.medium.com/convolutions-and-backpropagations-46026a8f5d2c> - Date of access: 10.05.2024.
3. Нейросети для работы с последовательностями [Electronic resource]. - Mode of access: <https://education.yandex.ru/handbook/ml/article/nejroseti-dlya-raboty-s-posledovatelnostyami> - Date of access: 10.05.2024
4. Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, Illia Polosukhin. Attention Is All You Need // arXiv preprint arXiv:1706.03762. – 2023 – С. 2-6.
5. Трансформеры [Electronic resource]. - Mode of access: <https://education.yandex.ru/handbook/ml/article/transformery> - Date of access: 10.05.2024.
6. Jacob Devlin, Ming-Wei Chang, Kenton Lee, Kristina Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding // arXiv preprint arXiv:1810.04805. – 2019 – С. 3-5.
7. Rico Sennrich, Barry Haddow, Alexandra Birch. Neural Machine Translation of Rare Words with Subword Units // arXiv preprint arXiv:1508.07909. – 2016 – С. 3-4.
8. Wordpiece tokenization [Electronic resource]. - Mode of access: <https://huggingface.co/learn/nlp-course/chapter6/6> - Date of access: 10.05.2024.
9. Job Salary Prediction [Electronic resource]. - Mode of access: <https://www.kaggle.com/c/job-salary-prediction/data> - Date of access: 10.05.2024.
10. HeadHunter API [Electronic resource]. - Mode of access: <https://github.com/hhru/api/> - Date of access: 10.05.2024.